reverse engineering Robotron 2084 for the Apple II



fschuhi

Posts: 38
Joined: 23 Feb 2019

reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Tue Feb 26, 2019 10:40 am

With the proliferation of really good emulators for the Apple II, I have recently been re-introduced to the games I played decades ago. Then, in school, there were 6502 wizards who not only cracked the copy protection on those games but were actually able to understand and change them. Awesome! Since then it has always bugged me that I shied away from learning 6502 and diving into 8bit game design. I don't know why I have finally decided to try this myself, maybe it was just a critical mass thing. Anyways, here I am, all set.

I chose Robotron 2084 as a target for reverse engineering because I play it reasonably well and have very fond memories. An added advantage which might come in handy at a later point in the project is that Robotron is a well-researched game. There is 6502 source code available for the Williams coin-op system (where the game was officially launched) as well as for the Atari7400 and other platforms - - but not the Apple II.

From reverse engineering projects in other domains I know that it is never recommended to try to understand a target system completely. This rule certainly applies here, because
a) I basically don't know anything about the 6502
b) I owned an Apple II in the 80s but never understood the hardware.
c) I don't have a clue about game design

Apart from those pretty damning constraints, Robotron is a pretty big game, with a lot of stuff happening on screen. I have to be prepared that the inner workings of the software are pretty arcane and convoluted.

The starting point for the project was James Tauber's Apple II emulator (http://jtauber.com/applepy), written in Python. At this point I should add another one:
d) I am a Python newbie

Python knowledge is needed for a different project, expected to start towards the end of 2019. I think it is easier to learn a language with a good project and right now I can't think of any project being a better one than reverse engineering a 6502 game! 

Reverse engineering generates a lot of data which has to be collected and correlated as easily as possible, which maybe makes the choice of using Python a bit more sensible. Note that I am not interested in a high fidelity recreation of Apple II artefacts, nor execution speed - - to achieve both I use the excellent https://en.wikipedia.org/wiki/AppleWin.

I had to port ApplyPy to Python 3, but else nothing else was needed to do a "first light" with running Robotron from the ram image.

The last couple of days I did the following:

- * read relevant sections of the Apple II Reference Manual (Woz, 1979)
  * understand 6502 opcodes and addressing modes
  * collect execution traces
  * collect info on memory locations (leap from/to info, touch counts etc.)
  * disassemble code with automatic labels and other leap info (who calls what)
  * intercept self-modifying code
  * link the emulator to Excel (via https://www.xlwings.org/
  * save/load complete state of 6502, memory, as well as Apple II display and softswitches

For now I focus exclusively on the splash screen part of the intro section of the game, that's difficult enough for its 600 lines of disassembled 6502 code.

The immediate goal is to map the execution flow for this very first part of the game. So far that has turned out considerably more difficult than expected, mainly because of "subroutines" PLA'ing the return addresses from the stack (i.e. mixing JSR and JMP), also PHA'ing different addresses and doing RTS more as a jump than as a return from subroutine. The whole idea of statically analysing the 6502 code seems a bit flawed, or it has to be at least complemented with runtime info.

Still, it has been possible to identify a bunch of "compact" subroutines: a contiguous stretche of executable 6502 code, leaped into the top by 1 or more JSRs, exit via normal RTS, no jumps from anywhere else into the middle of the stretches - - what I would call a "subroutine" in the normal sense. Call trees from those compact subroutines are directed acyclic graphs, which in turn will help me understand the execution flow.

The next step will be to establish which stretch of the code reads and writes certain memory locations, together with the particular addressing modes. Currently I am far too weak in 6502 to try to understand the code itself, just by reading it. I need more hints on what a stretch of instructions does. Of particular interest are the indirect-index / index-indirect instructions, because they shed light not only on the organisation of the overall memory but also on the meaning of zeropage addresses.

At this point I am looking for general advice on reverse engineering 6502 code.

- * How to generate sensible call trees for code which uses RTS as tabled jump facility?
  * Are there reverse engineering tools for 6502 code?
  * Are there any good books or other documention about reverse engineering 6502 out there?
  * What are common pitfalls when analysing 6502 code?
  * What are structures to look out for?
  * Is there value in trying to identify blocks of instructions and grouping them into macros, to speed up the understanding?

Thanks in advance for any pointers!

I intend to come back to this forum topic from time to time

and add info on my progress. The project could very well be too ambitious for me, but I am looking forward to the challenges on the way.



fschuhi

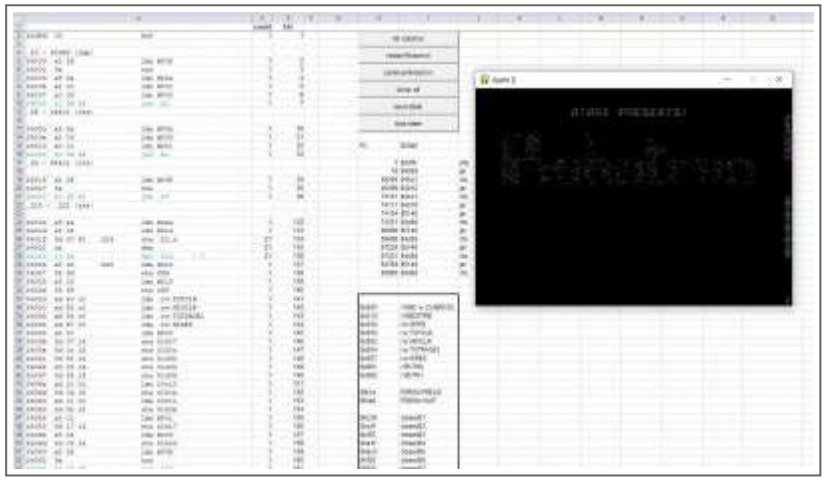
Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

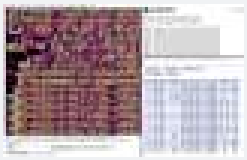
by **fschuhi** » Tue Feb 26, 2019 10:51 am

Here 2 images, one from AppleWin, showing the very first part of the Robotron intro, the other one a view on the Excel workbench - - just to give you an idea about what I am working with right now.

ATTACHMENTS







BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Tue Feb 26, 2019 12:24 pm

A grand adventure, and some interesting techniques. You've already met with the creative use of the stack: pushing and RTS is often used for a table-driven dispatch.

Contact:

The only thing I can think of to look for is that sometimes conditional branches can be proved to be always taken, so they become branch-always - an opcode found on the 65C02 but not the 6502.

It's unusual to see more than one TXS in a program. I suppose that can work as a setjmp/longjmp, or a cold vs warm restart.

Very rarely you will see a tactic of arriving mid-way through a multi-byte instruction, such that the operand becomes an opcode. It can save a cycle, or a byte, or something, because it gives two paths to the code beyond it.

I think you're right to try to see how zero page is used - you'll find single bytes as values, pairs of bytes as values, and also find pairs of bytes acting as pointers.



fschuhi

Posts: 38
Joined: 23 Feb 2019

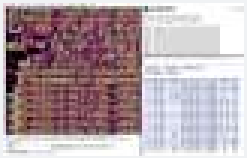
Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Tue Feb 26, 2019 1:09 pm

That's interesting - - there are 4 TXS occuring in the Robotron code, 3 of visible in the screenshot, i.e. right at the beginning of the program. The code modifies itself by overwriting the initial part w/ NOP (not shown, I intercept this), so that's in line with your remarks on cold vs warm restart. In all 4 cases the stack pointer is set to #$3F, so it could be a protection scheme for variables on the stack page. Thanks for the hint.

> **Quote:**
> Very rarely you will see a tactic of arriving mid-way through a multi-byte instruction,

I've added a check to the emulator right away, to get a warning when the pc is set to a byte which I know served as operand for a previous operation. So far it looks like Robotron's developer has not used this trick. Unbelievable what you can do with the 6502



BigEd

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Tue Feb 26, 2019 1:19 pm

Oh, yes, self-modifying code - that can happen! Including the case of copying some code to zero-page and

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

executing it, where it can self-modify itself even more efficiently.



whartung

Posts: 1004
Joined: 13 Dec 2003

Re: reverse engineering Robotron 2084 for the Apple II

by **whartung** » Tue Feb 26, 2019 4:28 pm

> **fschuhi wrote:**
>
> There is 6502 source code available for the Williams coin-op system

That's a real trick, being as Williams uses the 6809.

> **Quote:**
>
> Python knowledge is needed for a different project, expected to start towards the end of 2019. I think it is easier to learn a language with a good project and right now I can't think of any project being a better one than reverse engineering a 6502 game!

Personally, I learn a language by writing code, not reading it.

What are your overall goals from this project? What are you trying to learn, and what are you expecting this to teach you?

> **Quote:**
>
> At this point I am looking for general advice on reverse engineering 6502 code.
>
> - * How to generate sensible call trees for code which uses RTS as tabled jump facility?
> * Are there reverse engineering tools for 6502 code?
> * Are there any good books or other documention about reverse engineering 6502 out there?
> * What are common pitfalls when analysing 6502 code?
> * What are structures to look out for?
> * Is there value in trying to identify blocks of instructions and grouping them into macros, to speed up the understanding?

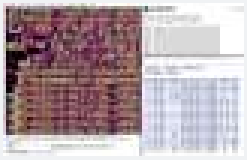
Doesn't White Flame have a tracing disassembler on the web? That might be a start.

Before understanding the robotron code, you best understand the architecture of the Apple II. Notably things like the video hardware, the hows and whys of VBLANK, and screen syncing.

At a basic level games are broken in to two phases. The "doing" code, and the "drawing" code. "Doing" is the reading of controls, moving objects, checking for collisions, etc.

"Drawing" is rendering the graphics on the screen.

VBLANK is when the TV beam is moving from the bottom of the display back up to the top of the display. Many games do the bulk of the their drawing code during this time, because the developers know that whatever changes they make won't be visible. Otherwise, they are "racing the beam". Simply, there's an intimate relationship between what the electron beam of the monitor is doing and what the game is doing, so it's best to understand how that relationship is manifest on an Apple II (I have no idea), because then you can see how the code interacts with it. Many simple games did ALL of their work during VBLANK. But Robotron won't be simple.



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Tue Feb 26, 2019 4:50 pm

> * Is there value in trying to identify blocks of instructions and grouping them into macros, to speed up the understanding?

I'd suspect not. Good names for routines and for locations will be a big help, and note that you might need to rename them too, as you figure things out. It's also possible that locations will be time-multiplexed so one location serves several temporary purposes.

The Apple II is relatively simple though, which will help. No interrupts, no custom chips with autonomous behaviour. The layout of video memory is non-obvious, and so any high-performance routines to update video memory may be quite clever.



Chromatix

Posts: 1462
Joined: 21 May 2018

Re: reverse engineering Robotron 2084 for the Apple II

by **Chromatix** » Tue Feb 26, 2019 5:33 pm

From a structural point of view, you may find that classifying subroutines into four classes will aid understanding:

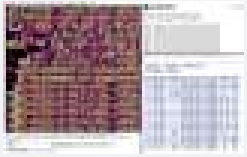
1: Where there is no stack manipulation (PHA/PLA/PHP/PLP/TXS) between entry and RTS. You've already done this, I think. (In 65C02 code, you'd also look for PHX/PHY/PLX/PLY.)

2: Where there are pushes balanced by pops later in the same routine, and no manipulation of the stack beyond the level at entry.

3: Where there are pushes *not* fully balanced by pops before RTS, or return is via an RTI instead of RTS. This is one sign of "clever code" which will need deeper analysis.

4: Where there are pops of the return address *from* the stack. This is normally done to access "inline data" placed immediately after the entry JSR. Expect the address to be transferred to zero-page, incremented during access, and then used in an indirect-JMP for return. You should be able to decipher (or use trace data to observe) where the return actually ends up. The gap between there and the JSR will contain inline data which may be revealing (it may be ASCII text).

One good use of the fourth category is in Woz' Sweet16 bytecode interpreter, which I understand was included in the Apple][ROMs. There are listings and a discussion of Sweet16 in our documentation archives. If you encounter a call to Sweet16, you can decode the opcodes directly instead of puzzling over the interpreter itself.



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Tue Feb 26, 2019 5:39 pm

Another thought, although I'm not sure what Apple II offers by way of a BRK handler, but it's possible you'll see the BRK opcode used as a software interrupt.



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Tue Feb 26, 2019 7:33 pm

> **whartung wrote:**
>
> > **fschuhi wrote:**
> >
> > There is 6502 source code available for the Williams coin-op system
>
> That's a real trick, being as Williams uses the 6809.

Just checked again, and of course you are right, that doesn't look like 6502 at all. The Atari code does, though, so at least that one might come handy. I wouldn't be surprised if it doesn't. The Apple II port was obviously done by Atari (at least it says so in the intro), so I might get something out of it when I try to understand certain parts of the Apple code.

> **Quote:**
>
> Personally, I learn a language by writing code, not reading it.

Yes, me too. Still, I like the idea of combining learning Python and learning about the 6502, plus learning about game design, using one of my favorite games from ages ago.

> **Quote:**
>
> What are your overall goals from this project? What are you trying to learn, and what are you expecting this to teach you?

I chose the "Nostalgia" board for a reason. It's a bit about revisiting the past and doing something I did not do then. The overall goal is therefore a bit fuzzy, and that's ok for now. At the same time I am really at awe when I see what developers are able to cram into, well, about 13k in the case of Robotron. There are a lot of lessons to learn, I'm sure.

> **Quote:**
>
> Doesn't White Flame have a tracing disassembler on the web? That might be a start.

I checked that out today, did also some reading in the WFDis thread on this board (thanks to White Flame pointing this out to me). That's for sure a great piece of software.

I'm working from a different position, though. WFDis is doing static analysis of the code, guided by manually hinting to where the code might be. I have it easier: I run the code with an emulator, so I know with certainty where the opcodes and their operands are. I am also able to resolve a lot of cases which are difficult (if not impossible) to infer from the code alone, e.g. detecting RTS jump tables and other stack-based wizardry. Because I know everything about the execution flow it is easy to annotate the assembly listing.

From one post from White Flame I have gleaned a nice idea, though. WFDis uses a "flooding" algorithm to try to detect the instructions to disassemble. This idea can be used to detect relocatable code, or what I called "compact subroutines" in the OP. Those are important building blocks and it's good to pinpoint them as early as possible.

I will have to dive deeper into WFDis, though, thanks for bringing this up.

> **Quote:**
>
> Before understanding the robotron code, you best understand the architecture of the Apple II. Notably things like the video hardware, the hows and whys of VBLANK, and screen syncing.

Great feedback! I have already downloaded a number of books on Apple II hardware and software. Particularly the display is a bit messy, due to weird addressing schemes of the colored hires pixels. Operating on sprite data directly in the display memory would be an algorithmic nightmare. But for the purpose of reverse engineering that's actually quite nice - - It's a certainly a bet, but I think the odds are in my favour that the developer of Robotron stored the sprite data somewhere else, in a more legible format. I already know the parts of the code which are responsible to throwing the stuff onto the screen, by watching which parts of the code STA to pages $20 and following. I don't know but I would guess that I'll find the LDA to the sprite tables in the vicinity.

It is certainly not the normal way to understand a target

platform, you are right, but it might turn out to be a fun excercise. And don't get me wrong, it's certainly going to be much harder than I just outlined!

At least it looks like Robotron doesn't make use of VBLANK info, as it doesn't access the byte which indicates that the Apple II is in the vertical retrace region. Maybe I don't have to count cycles then.

> **Quote:**
> But Robotron won't be simple.

Yeah, thanks for reminding me



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Tue Feb 26, 2019 7:39 pm

> **BigEd wrote:**
> Another thought, although I'm not sure what Apple II offers by way of a BRK handler, but it's possible you'll see the BRK opcode used as a software interrupt.

Thanks for making me aware about that. I think BRK on an Apple II does a hard reset - - it's of limited value on that platform. The Robotron code only uses BRK in one place: To close the door at the beginning of the code, so that the original init section is definitely never executed twice (i.e. overwrites the JMP at the top of the loaded image w/ a BRK).



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Tue Feb 26, 2019 7:56 pm

> **Chromatix wrote:**
> From a structural point of view, you may find that classifying subroutines into four classes will aid understanding:

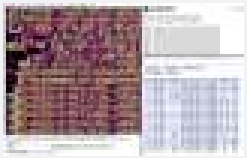
Thank you, Chromatix, that's really good food for thought. I've actually identified your first 3 cases in the code, by checking the pairings between JSR'd stack addresses and what the next RTS finds on the stack. It is most often what can be expected from a "compact" subroutine. But not always:

> **Quote:**
>
> This is one sign of "clever code" which will need deeper analysis.

I'm a bit torn: should I focus on pulling the clever code apart, or focus on the easier (i.e. more "compact") code first? Right from the start, just in the first couple of dozen instructions there is some cleverness going on in the execution trace, obviously some initialisation code. I would like to have a organised approach for such sections, but that might not be possible. There will be code where I will have to decipher the execution flow manually.

> **Quote:**
>
> 4: Where there are pops of the return address *from* the stack. This is normally done to access "inline data" placed immediately after the entry JSR. Expect the address to be transferred to zero-page, incremented during access, and then used in an indirect-JMP for return. You should be able to decipher (or use trace data to observe) where the return actually ends up. The gap between there and the JSR will contain inline data which may be revealing (it may be ASCII text).

That's exciting! Looking at the disassembly, it seems there are a number of candidate cases where something like this might be happening. The Robotron code does not contain any indirect JMPs, so I should expect RTS doing the trick. I'll keep this in mind.



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Tue Feb 26, 2019 8:07 pm

I think for the most part you'll be able to understand the clever code so long as you can zoom out far enough to see the whole picture of what it's doing. If you do find anything you can't get your head around, I'm sure there are many here who would be more than pleased to have a 6502 puzzle to solve.

BTW, you might like this diagram of the 6502's architecture and instruction set:
viewtopic.php?p=42766#p42766



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Tue Feb 26, 2019 8:08 pm

> **BigEd wrote:**
>
> > * Is there value in trying to identify blocks of instructions and grouping them into macros, to speed up the understanding?
>
> I'd suspect not. Good names for routines and for locations will be a big help, and note that you might need to rename them too, as you figure things out. It's also possible that locations will be time-multiplexed so one location serves several temporary purposes.

Well, I am usually tempted to turn everything into a domain-specific language. I should put more thought into a good annotation system, to mix automatically generated disassembly (containing trace / watch info) w/ my notes and comments. Thanks for your advice.

> **Quote:**
>
> The layout of video memory is non-obvious, and so any high-performance routines to update video memory may be quite clever.

I fear there will be a rude awakening from the current honeymoon period. Trying to understand what's happening in those video memory update routines might actually be too difficult for me, but it's worth a try.



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Tue Feb 26, 2019 8:20 pm

> **Quote:**
>
> BTW, you might like this diagram of the 6502's architecture and instruction set:

This site and this forum contains so much info, I'd like to drop out of work (and life in general) for a while and just read!

Is there a consensus on which 6502 book is the best? In school we were pointed to Zaks' "Programming the 6502", but it's quite verbose. Any love for Scanlon's "6502

Software Design"?

... Probably not the right place to ask this question. Which board should I check?

Post Reply 98 posts 1 2 3 4 5 ... 7

Return to “Nostalgia” Jump to

Forum Home Search Contact

All times are UTC

6502.org is an ongoing project by Mike Naberezny and contributors. About · News · GitHub

reverse engineering Robotron 2084 for the Apple II



Chromatix

Posts: 1462
Joined: 21 May 2018

Re: reverse engineering Robotron 2084 for the Apple II

by **Chromatix** » Tue Feb 26, 2019 8:35 pm

Cleverness in the very early stages of a game may be part of a copy-protection scheme, checking for example that it's been loaded at the correct address and verifying a simple checksum. Such things are easy to defeat by overwriting with NOPs, as long as they don't induce side-effects elsewhere, and this is often done to port them to environments that are almost, but not entirely, like the one originally intended.

I would advise looking at a "leaf function" first, perhaps one of the subroutines that does nothing or only simple things with the stack, and try to work out what it does. Especially you want to look at leaf functions that hit frequently at runtime (since you have traces, you can identify those easily). You might not get much of a high-level overview at first, but the gestalt will begin to emerge over time.

You could also statistically correlate accesses to memory areas by sections of code. Do this at the byte level for zero-page and device addresses, and the page level (high byte of the address) elsewhere, then drill down as required. For zero-page in particular, you might distinguish between addressing modes used for access, as indirect accesses signify use of that location as a pointer, while direct accesses may just be for temporary storage (but also for updating pointers). For example, this should help you identify code and zero-page pointers associated

with the screen drawing process.

Also pay attention to subroutines that call into the ROM. Those routines will be documented in Apple literature and offer more clues as to what is intended by the game's programmer. Modify your tracer to log the parameters passed to (and maybe results returned from) these standard routines, while skipping the actual instructions making them up.

Last edited by Chromatix on Tue Feb 26, 2019 8:54 pm, edited 1 time in total.



cbmeeks

Posts: 1254
Joined: 17 Aug 2005
Location: Soddy-Daisy, TN USA
Contact:

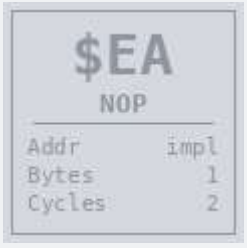
Re: reverse engineering Robotron 2084 for the Apple II

by **cbmeeks** » Tue Feb 26, 2019 8:46 pm

FWIW, the comp.sys.apple2 newsgroup is an excellent place for all things Apple II. Probably even better than the AtariAge forum (which can also be good).

Those guys really know the A2 game scene. Of course, for expert level 6502 information, you can't beat this place.

Cat; the other white meat.



whartung

Posts: 1004
Joined: 13 Dec 2003

Re: reverse engineering Robotron 2084 for the Apple II

by **whartung** » Tue Feb 26, 2019 9:01 pm

> **cbmeeks wrote:**
> FWIW, the comp.sys.apple2 newsgroup is an excellent place for all things Apple II. Probably even better than the AtariAge forum (which can also be good).
>
> Those guys really know the A2 game scene. Of course, for expert level 6502 information, you can't beat this place.

It may very well be worthing pinging those groups to see if anyone has done this already, if for no other reason than to get a leg up.

I would focus on the largest, clearest chunks of code as I can to (ideally) eliminate them from the prospective code base of "unknown code". The more you know earlier, the better it will go, and it can very well help in understanding

the complicated code.

One of the hardest, yet in many ways very important, things to suss out is the use of the global variables, since those will likely point to the important internal data structures. From there you can work on the code the manipulates those structures, and then the code above that.

You can also start at the inputs (I have no idea how Robotron worked on the Apple II), and go from there. What's reading the joysticks? What do they do with the data?

Lots of different approaches to poking the black box.



Chromatix

Posts: 1462
Joined: 21 May 2018

Re: reverse engineering Robotron 2084 for the Apple II

by **Chromatix** » Tue Feb 26, 2019 9:12 pm

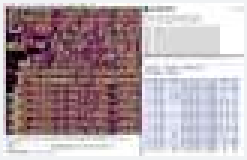
A note about addressing modes - these can broadly be divided into three major categories on the 6502:

1: Implied and Immediate - nothing happens outside of the program bytestream. Arguably the stack instructions also go here, though their effect is also bounded.

2: Zero Page, Absolute, and Indexed. These are "direct" accesses; an address is read from the program bytestream, possibly modified by the X or Y index registers, and used directly to locate the operand. Indexing is always by the X register except for LDX and STX, which can index on the Y register instead. These are reasonably fast memory accesses, especially to Zero Page since one less byte has to be loaded.

3: Indirect modes. For JMP this is a non-indexed indirection through a vector located anywhere, and thus referred to by a 16-bit address. For all other instructions there is a pre-indexed indirect in which a zero-page address is modified by the X register to locate the pointer to the operand, and a post-indexed indirect in which the pointer in zero-page memory is modified by the Y register. These are quite slow accesses since the CPU must access several bytes of memory and perform pointer arithmetic just to locate the operand - however the flexibility is very valuable.

Self-modifying code might seek to increase speed by replacing the operand bytes of instructions to gain the flexibility of Indirect addressing modes without the performance penalty normally associated with them. Detect this by watching for writes to addresses that have previously been executed.



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Tue Feb 26, 2019 9:25 pm

> **fschuhi wrote:**
>
> Is there a consensus on which 6502 book is the best? In school we were pointed to Zaks' "Programming the 6502", but it's quite verbose. Any love for Scanlon's "6502 Software Design"?

I've started a thread, over here. Let's hope!



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Wed Feb 27, 2019 8:22 am

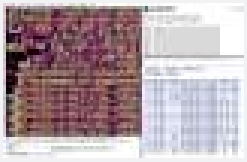
Thank you all for your advice! Really valuable, and motivating.

This is my plan:

- * *Study 6502 assembler.* I am not versed enough to read 6502, structures do not stand out yet. My reading list starts w/ "Assembly Lines" (Wagner 1982), because it's 6502 + Apple II centric, then "6502 Assembly Language Subroutines" (Leventhal 1982).
  * *Study the Apple II.* After browsing a couple of books I've settled on "Understanding the Apple II" (Sather 1985), together with the Woz-1979 reference manual.
  * *Stick to the Robotron game intro.* I'm in no hurry to get to the really difficult stuff. First I need to have a reasonably small workbench with simple and deterministic execution flow. In addition, from comp.sys.apple2 I learned about a patched version of Robotron with different intro screen elements and menus. This will be helpful to guide my understanding.

* *Derive a (graphical) call tree.* Needed for both understanding what the leaves do, as well as "zooming out" to see the bigger picture. I'll use Graphviz to picture the tree.
* *Collect memory info for subroutines.* One of the recurrent themes from your posts is the importance to understand memory access, both direct and indirect. I'll also try to devise ways to automatically identify global variables.

This should keep me occupied for a while. I'll resurface when I have something tangible to show.



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Wed Feb 27, 2019 8:44 am

Great - do keep us updated with progress!

(The Apple II disk system is another twisty maze with marvellous possibilities, but hopefully you can sidestep all that, if the program just loads once and doesn't use the drive again.)



White Flame

Posts: 704
Joined: 24 Jul 2012

Re: reverse engineering Robotron 2084 for the Apple II

by **White Flame** » Wed Feb 27, 2019 1:36 pm

Can you post a binary of the specific Apple II program version you're using, so we can look at it as well?

Regarding WFDis, it is an in-progress front-end prototype with a simple tracing disassembler included. There's a smarter back-end server that does heavier analysis, but is waiting on new front-end features to support it better.

The biggest advantage I see in an interactive disassembler like this is the ability to rename things on the fly. Applying a label, then seeing all instances of the code using it turn from "LDA $81" into "LDA playerAnimFrameVector + 1" is a huge help in interactively exploring the codebase, even if the names are temporary and hold partial understanding (they can always be renamed just as easily). Plopping in comments and being able to follow address references via hyperlinks and back/forward browser history is really nice. Even with the relatively basic feature set in there now, I find this workflow incredibly quick.

Discovering code vs data is actually quite a small part of the work, though it's nice when it's automatic. Figuring out *why* a piece of code/data is in there and what it's doing is the real reverse engineering.

WFDis Interactive 6502 Disassembler
AcheronVM: A Reconfigurable 16-bit Virtual CPU for the 6502 Microprocessor



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Wed Feb 27, 2019 8:35 pm

I've downloaded the version I use from https://www.myabandonware.com/game/robo ... Apple%20II

I pulled the image from the disk with CiderPress, it's ROBOTRON.BIN, to be loaded and started at $2DFD.

White Flame, I've dabbled a bit with your WFDis, it's really impressive, and easy to use. I actually took the feature of on-the-fly renaming as an inspiration for my workbench. That workbench runs in Excel, which might sound weird but it's actually a really nice free-form way to work with different kinds of data. Supplying labels is of course not as WYSIWIG as WFDis, but reasonably convenient for now.

Please keep in mind that I run the code in an Apple II emulator. This gives me a lot of ways to hook into the CPU and the memory. It also enables me to distinguish RTS from PHA/PHA/RTS, not by looking at the code but simply running it and continuously inspecting the stack.

> **Quote:**
> Discovering code vs data is actually quite a small part of the work, though it's nice when it's automatic.

With an emulator the execution path determines what to disassemble, of course. I wouldn't even call it "automatic", the disassembly is just a byproduct. Where it gets difficult, though, is to unentangle the execution flow itself. Which parts do what? I wouldn't be able to answer this question by tearing into the 6502 disassembly, because I'm simply not good enough in 6502. When I get more proficient then the WFDis frontend could actually become valuable for me.

From what I've gleaned from a comment in

comp.sys.apple2, Robotron might not be easy to reverse engineer. From the few things I've learned so far, it doesn't look like intentional obfuscation. But already the intro section is really convoluted, with parts of subroutines are stiched together with JMP, there is some recursive calling, tabled RTS, self-modified code, moving around of data, code is executed in weird places.

I've been able to learn about all this by collecting runtime info and doing some first analysis on it. So far the algorithms for this have been adhoc. From the feedback I got in this thread I decided to focus first on "call trees" for the 6502. I'm currently working on a concept and first prototypes, but it'll take some time before I resurface. In the meantime it would be awesome if you shared some of your reverse engineering know-how, maybe even running your backend on the Robotron binary?



cbmeeks

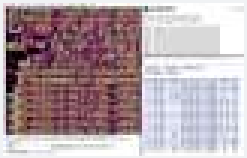
Posts: 1254
Joined: 17 Aug 2005
Location: Soddy-Daisy, TN USA
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **cbmeeks** » Wed Feb 27, 2019 9:08 pm

FYI, over on comp.sys.apple2, there is a user named qkumba who is an absolute legend in porting Apple II games from DOS to PRODOS. He has done literally hundreds. He might be a good resource on learning the tricks to Robotron's disk system.

Cat; the other white meat.



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Thu Feb 28, 2019 8:52 am

> **White Flame wrote:**
>
> Applying a label, then seeing all instances of the code using it turn from "LDA $81" into "LDA playerAnimFrameVector + 1" is a huge help...

Really good example: an operand byte might be best viewed as an expression, which you'll only be able to conjure up as you get to appreciate the code. It's not too unusual for an index not to be zero-based - if the valid values for an index are 10-25 then the index might well be used to point a little below the data in question.

Another 6502 feature: in most situations, zero page addresses wrap around within zero page. Therefore a negative byte does act as a negative offset. (Being very clear about byte values and their interpretation either as signed or unsigned numbers, or as an enumerated type, or a character, or other things, could be an important watershed.)

Likewise, the idioms for performing arithmetic or comparisons on two-byte or multi-byte values, the idioms for using the carry bit as a boolean variable or as an accessory in multi-byte shifts, are important to understand. They are not exactly macros, but they are idioms larger than single instructions.



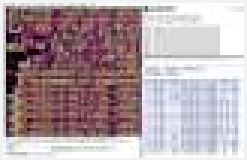
fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Sat Mar 02, 2019 1:14 pm

BigEd, your points show how important it is to understand the code not only broadly but also on the scale of what you call "idioms". I think that's also what White Flame alluded to, and I totally agree. Furthermore, understanding idioms just by reading them is much harder than using them in own software. That was a good point of Whartung in one of the earlier posts: it's easier to understand things by making them, not just reading about them. I'd like to spend my time efficiently, so I hope that there is some balancing point between such a "micro" approach, and the more macro of looking at the "whole picture".



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Sat Mar 02, 2019 2:01 pm

I'm a bit of a reader of code, myself! Studying some of Bruce's code, or Woz', or maybe Tom Pittman's TinyBasic, might be rather illuminating.

But if you want to write code, here's an idea: write a routine to clear a 1000 byte block of memory, the address being known at assembly time. You can probably write it three or four ways, and doing so might itself be a good exercise. In fact, this might be a good idea for a thread...

... Edit: here's the new thread.



sark02

Posts: 241
Joined: 10 Nov 2015

Re: reverse engineering Robotron 2084 for the Apple II

by **sark02** » Sat Mar 02, 2019 7:36 pm

I've had a background project over the past decade of disassembling the original Williams Defender code. I tend to work on this about 3 or 4 half-days a year, when I'm on vacation - between doing other stuff. So although a decade sounds like a long time (I think it might actually be 12 or 13), I've only just begun. It's not a serious thing, just a little fun when I'm absolutely completely bored.

Defender was written by Eugene Jarvis, the same guy who wrote the original Robotron 2084. The main game loop iterates over object lists: linked-lists of data structures which have references to sprite objects, "think" (behavior) functions, timers and other things (TBD!). Sprite objects reference pixel data, and have pointers to draw and erase functions. Then there's the think functions for the on-screen objects, input, overall game logic, etc.

The 6502 version of Robotron is likely going to be structured similarly, even if it's an independent reimplementation by a different programmer. This "object oriented" style is a pretty natural way of writing games with many different "actors" that have their own lifespan, behavior, and graphics/animation.

My approach to the big-picture disassembly was:
- Start with a two-pass disassembly that creates labels for all the trivial branch/jump destinations. This identified the 'direct-call' subroutines (called via JSR) and the intrafunction logic (control flow / loops).
- Make a list of all the JSR destinations. This is the "known function list".
- For each RTS instruction that isn't followed by a known function or an intrafunction label, take the address of the next byte and look through the binary for (LO-BYTE, HI-BYTE) references to that address. This becomes the "possible function list". If there a multiple bytes between the RTS and the next known-function, search for those too. The locations of the (LO, HI) references are "possible function pointers". Keep those.
- For each "possible function", disassemble it and see if it looks like garbage or an actual function. If you know 6502 then it's pretty easy to tell the difference.

One difference between 6809 and 6502 is that 6809

natively handles 16-bit values, so (LO, HI) pairs are often directly stored in contiguous memory addresses. When I used to write small games on the 6502 (back in the early 80s) I would spread object structure fields over arrays indexed by X. So, for example:

CODE: SELECT ALL

```
struct object {
    struct object *next;
    uint8 xpos;
    uint8 ypos;
    uint16 thinkfunc;
};
```

In 6809 this would be a 6-byte object (2 bytes for the 'next' pointer), whereas in 6502 I might have structured it like this:

CODE: SELECT ALL

```
obj_next:     .ds 128   ; support 128 objects
obj_xpos:     .ds 128
obj_ypos:     .ds 128
obj_thinkhi:  .ds 128
obj_thinklo:  .ds 128

obj_head: .ds 1    ; head index

obj_runloop:
    lda   obj_head
obj_loop:
    tax
    ;  call the think function
    lda   obj_thinklo,x
    pha
    lda   obj_thinkhi,x
    pha
    rts
objloop_next:    ; think function JMPs back to here
   lda   obj_next,x
   bne   obj_loop
   rts
```

If 6502 Robotron structures things like this then finding these (LO, HI) pairs might be a challenge. You can find loops like the above example to find the arrays, and then adjust your analysis accordingly.

With the known-function-list you can create a call-graph. There will likely be many call graphs - they won't all neatly form one big tree. The call-graph lets to see structure. There will be many shared nodes on the graph. These help identify utility functions.

- Using a graphical utility, locate the sprite patterns for the program. I wrote my own utility for this, by studying how Defender draws its sprites and therefore what order the data had to appear in memory. Different sprites are different widths and heights, so the utility was interactive I could move though memory and adjust the x and y strides. Find all the sprite data and record the start address of each pattern.

- Find all the draw and erase functions. Defender did not use a double-buffered screen, rather it would erase an object from its old position just before drawing it at its new one. The draw and erase functions in 6809 are pretty neat as they use two stack pointers to efficiently move memory in blocks.

- Reference to the draw/erase functions should be in your (LO, HI) function pointer lists, so with that you should be able to find the sprite descriptors (data structures that describe a sprite, what it looks like, how wide and high it is, and where its draw/erase functions are).

- Find the object lists. Objects in Defender come in different sizes and are allocated and free using utlity functions. Want a new alien? Call the allocate function for a "type 1" object, and then fill in its fields. The object comes to life on the next iteration of the main loop.

- Find the hardware manipulations. These are reads/writes to hardware registers, which you find out by studying the Apple II reference manuals. Every time there's a register read, understand what that does. Every time there's a write, know what it does. It is changing a color register? Is it starting a timer? Is it clearing an interrupt? Is the read getting a joystick value? A key value? Find all these and try to understand their enclosing function.

- Find the interrupt handlers. If the Apple II uses interrupts then find the vector and start trying to figure out its big-picture. Interrupt functions end with RTI, but the Apple II ROM may take care of that and the interrupt might be invoked via a software vector. Go read that vector and see where it points to.

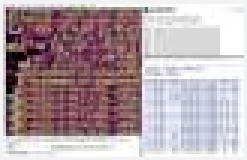
Being able to BREAK into an emulator makes a lot of this easier as you can see the state of the machine while the

game is actually running. For Defender, the MAME emulator was invaluable in this initial study as the game uses ROM paging, and just figuring out which banks are the game proper vs. the menus and diagnostic pages wouldn't have been easy without prior knowledge.

The hardest thing with reverse-engineering is figuring out intent. Not _what_ something does, mechanically, but _why_. Why is the code comparing this to that? What's the significance of the literal value #$CE in this CMP? With an emulator you can change these values and see if you observe a difference. That's playing on hard mode, for sure... but you can do it.

I hope this gives you some ideas. Some people have a natural talent at reverse-engineering. I'm not one of them, and I'm sure I'll never make significant progress with Defender, but hey... everyone needs a hobby... even if it's only a handful of hours a year.

Good luck!



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Sat Mar 02, 2019 7:40 pm

Oh, that's a good point, sark02, the use of two tables for the high and low bytes of pointers (or other values) so the same index can be used for both bytes – that's a fairly common idiom, and not an obvious one.

Post Reply 98 posts 1 2 3 4 5 ... 7

Return to "Nostalgia" Jump to

Forum Home Search Contact

All times are UTC

reverse engineering Robotron 2084 for the Apple II



sark02

Posts: 241
Joined: 10 Nov 2015

Re: reverse engineering Robotron 2084 for the Apple II

by **sark02** » Sat Mar 02, 2019 10:20 pm

> **sark02 wrote:**
>
> Some people have a natural talent at reverse-engineering.

If anyone is interested in Williams Robotron and its object management, a chap called Scott Tunstall has done a very detailed disassembly of the code.
Find it via Sean Riddle's (very interesting) website:
https://seanriddle.com/robomame.asm



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Sun Mar 03, 2019 1:26 am

sark02, fantastic info, thanks for taking the time. I really appreciate your help. This is so motivating! I will certainly come back to your post many times.

I have peeked into the game for a couple of days now. My understanding is, well, limited - - even more so because I've never coded the 6502 so far. I'd still like to give you some feedback on how I work my way towards the "big picture".

> **Quote:**
>
> My approach to the big-picture disassembly was:

My disassembly follows the execution path, i.e. I decide on the basis of actually executed instruction where the opcode, the operands and the data are.

At this point in time my disassembler creates the following file:

CODE: SELECT ALL

```
.L0 < $2dfd (jmp)
$4000  a2 3f                  LDX #$3f
$4002  9a                     TXS
$4003  a9 2e                  LDA #$2e
$4005  a2 0c                  LDX #$0c
$4007  a0 08                  LDY #$08
$4009  20 9d 44               JSR .closed01
.L4 < $44c1 (rts)

$400c  a9 3a                  LDA #$3a
$400e  a2 06                  LDX #$06
$4010  a0 01                  LDY #$01
$4012  20 9d 44               JSR .closed01
.L5 < $44c1 (rts)

$4015  a2 3f                  LDX #$3f
$4017  9a                     TXS
$4018  20 2f 51               JSR .closed13
.L21 < .L20 (rts)

$401b  a9 ea                  LDA #$ea
$401d  a2 14                  LDX #$14
```

The disassembler assigns labels to every point where we leap to (branches, jsr, absolute/indirect jmp, rts). Consequently, I can show the addresses from where I leap to a label w/ "<", together with the type of leap instruction. RTS which do not match a calling JSR (e.g. stack was manipulated) are treated as an own type of leap. The branch instructions are marked depending if the branch was sometimes, always or never executed.

I then take the code and group consecutive instructions. The code (seen as a list of consecutive instructions in memory) is broken into "tiles" at instructions which leap away or instructions which are leaped to from elsewhere. Tiles are then connected automatically using rules, e.g. for sequential execution, normally returning JSR, branch instruction which never branch but which could etc.

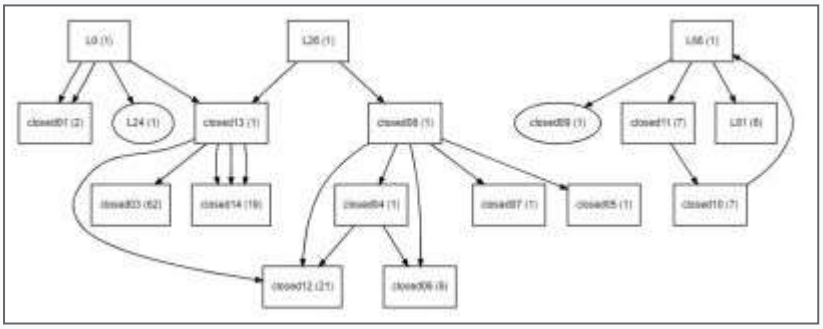
Those "stretches" of tiles are a first pretty good approximation of subroutines. I collect the JSR in each stretch and collect to which other stretch/subroutine they

leap and return.

So far I have emulated and disassembled only a couple of seconds into the intro of the game. The disassembly therefore has a lot of "holes", because certain parts of the intro have not been executed yet. Still, the game intro runs, so there has to be some structural logic, even if the coverage of code by the execution path is not complete. I'd rather take it slowly in the beginning

> **Quote:**
>
> With the known-function-list you can create a call-graph. There will likely be many call graphs - they won't all neatly form one big tree. The call-graph lets to see structure. There will be many shared nodes on the graph. These help identify utility functions.

I know of course what's happening in the game, but I am not yet able to relate it to the disassembly. No surprise there; a steep learning curve is to be expected. The structural information by executed the code is enough, though, to get a first glimpse on the call-graph.



The attached graph shows the flow between the stretches of the code, identified by the label of the first instruction of the stretch. The numbers show the count how often the stretch was entered (from the sample execution path). => The helper subroutines are clearly discernible, even with this very crude analysis.

The call-graph as well as the underlying stretches and their tiles are automatically generated from the memory map and disassembly information. The resulting call-graph covers most of the execution paths through the code. There are few locations which the algorithm cannot stitch together correctly, e.g. the cyclical calling in the right sub-call-graph. It offers enough information, though, to guide the next steps.

> **Quote:**

> The hardest thing with reverse-engineering is figuring out intent. Not _what_ something does, mechanically, but _why_.

I cannot even imagine how hard this is going to be. It will take ages until I will make good progress into the game. The info from your post and the posts of the others in this thread is a fantastic help, thanks!

ATTACHMENTS

into0.asm
(21.51 KiB)



Chromatix

Posts: 1462
Joined: 21 May 2018

Re: reverse engineering Robotron 2084 for the Apple II

by **Chromatix** » Sun Mar 03, 2019 3:13 am

Perhaps I can give you a head start by walking you through the subroutine labelled "closed03"; this has the highest call count in the above graph.
Please feel free to have this on in the background.

CODE: SELECT ALL

```
.closed03 < $5148 (jsr)
$4c4f  85 e0                STA $e0
$4c51  86 e1                STX $e1
$4c53  a9 00                LDA #$00
$4c55  85 e2                STA $e2
$4c57  a2 08                LDX #$08
$4c59  0a           .L8     ASL
$4c5a  26 e2                ROL $e2
$4c5c  06 e0                ASL $e0
$4c5e  90 07                BCC .L7       ; 0
$4c60  18           .L9     CLC
$4c61  65 e1                ADC $e1
$4c63  90 02                BCC .L7       ; +
$4c67  ca           .L7     DEX
$4c68  d0 ef                BNE .L8       ; 0
$4c6a  aa           .L10    TAX
$4c6b  a5 e2                LDA $e2
$4c6d  60                   RTS
```

One of the first things to note here is that the addresses shown are not a complete listing; two bytes at $4c65 are missing. I bet, if you examine the code by hand, they'll turn out to be **E6 E2** (INC $e2) - because this is an 8x8 multiplication routine, exactly the sort of thing you'd expect to be frequently executed as a leaf function, given a CPU without a multiply instruction of its own.

I think it should be made clearer in your listings when bytes are skipped, as there may be rarely-executed instructions or useful inline data there. Inline data can be particularly helpful for determining the purpose of a block of code.

The two 8-bit operands are provided in A and X, and the first thing the routine does is stuff them into zero-page at $E0 and $E1 respectively, then clear A and a workspace byte at $E2. The Y register isn't touched at all in the whole routine, so it is preserved for the caller to use as an index register of its own (which it does).

X is then initialised to 8 and used as a loop counter. The loop structure consists of the three instructions LDX #8 ; [...] ; DEX ; BNE .L8 (backwards branch) and encompasses the majority of the subroutine body. This is an idiom that you should learn to recognise instantly.

After the loop, note that X is not restored from $E1 (as it would be if the author intended to preserve it), but is instead transferred from A, and then A is loaded from the workspace bytes previously initialised. This indicates that both A and X are used to return data to the caller (in this case, a 16-bit product, with X holding the low-order half). We should try to work out what A and $E2 end up with inside the loop.

Returning to .L8, we immediately see ASL ; ROL $E2. This is a two-byte multiprecision left shift, effectively transferring the top bit of A into the bottom bit of $E2 via the Carry flag, and the former top bit of $E2 ends up in the Carry flag. This is further confirmation that they're being treated as a single wide value. But on the first iteration both are still zero, and remain so for the time being.

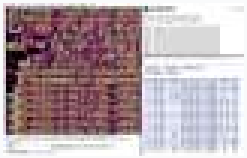
The following ASL $E0 is where the real action begins. Recall that $E0 contains the former value of A on entry; we've just popped out the top bit of it into the Carry. The very next instruction branches on the Carry, skipping all the rest of the loop body if it is cleared. Clearly the individual bits of the first operand, most-significant first, are crucial to the algorithm.

What happens when a bit is found to be set? Following on from .L9, we see the idiomatic pair CLC ; ADC $E1, adding the second operand to the accumulator. (You could reasonably contract CLC ; ADC pairs into ADD macros, and

SEC ; SBC into SUB - since the 6502 has no add-without-carry nor subtract-without-borrow instructions, those pairings are very common indeed.) So one operand is added to the shifted workspace for each set bit in the other operand; this is a classic multiplication algorithm!

The Carry flag now indicates unsigned overflow from the addition, and there is a branch testing it immediately afterwards - which happens to be always-taken for the moment. But the sensible thing to do if the Carry is set is to propagate it into the high half of the workspace at $E2, which can be done with INC $E2 as noted above.

Hopefully if you can follow the above explanation, much more will make sense from now on.



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Sun Mar 03, 2019 8:32 am

(Good point about the two missing bytes: it demonstrates that a tracing disassembler can only reach code which is executed, so we become interested in code coverage. It's not enough to run the game, it might be necessary to trigger every possible action and interaction, including every possible carry from one byte into two: so actions which are within 256 coordinate points, or 256 energy points, or 256 rotation units, need to be exercised until their range goes beyond one byte. Assuming they are not single-byte values, of course. It might be enough to annotate every branch which is always taken, or never taken, as something to be investigated manually.)



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Sun Mar 03, 2019 11:48 am

Chromatix wrote:

Please feel free to have this on in the background.

CODE: SELECT ALL

```
[/quote]
Mess with the best, die like the rest! :)

[quote]I bet, if you examine the code by hand, the
Yes! Nice.
```

```
Chromatix, you just jumpstarted my in-depth analys

[quote="Chromatix"]I think it should be made clear
[quote="BigEd"]It's not enough to run the game, it
The multiplication algorithm highlights that relyi

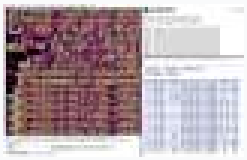
I have attached the complete disassembly of the bi

Now that I know that .closed03 is 8-bit multiplica

The multiplication algorithm on [url]https://www.l
```

ATTACHMENTS

Robotron (raw).asm
(373.16 KiB)



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Sun Mar 03, 2019 11:53 am

> I haven't started w/ a repository of 6502 idioms and algorithms yet, maybe it's time to contemplate that.

I can imagine that could be a very useful resource for future adventurers.



Chromatix

Posts: 1462
Joined: 21 May 2018

Re: reverse engineering Robotron 2084 for the Apple II

by **Chromatix** » Sun Mar 03, 2019 12:27 pm

> **Quote:**
> The multiplication algorithm on https://www.lysator.liu.se/~nisse/misc/6502-mul.html
> looks faster...

Yes, that's a nicer looking algorithm. It saves a branch and increment by sucking the overflow carry bit back into the accumulator as soon as it's produced, which in turn is made possible by processing everything LSB first instead of MSB first. But it's still fundamentally performing an add of one operand for each set bit in the other operand, so is recognisable as a multiply on the same basis.



sark02

Posts: 241
Joined: 10 Nov 2015

Re: reverse engineering Robotron 2084 for the Apple II

by **sark02** » Sun Mar 03, 2019 7:12 pm

If anyone is curious about what Robotron 2084 looks like on the Apple II, and how it compares to the arcade, see this "Let's Compare" video:
https://www.youtube.com/watch?v=ejA0w-PmBZY

The video starts with a clip from the Williams arcade version (6809-based, and uses a hardware blitter to draw the sprites; unlike earlier games like Defender, which drew sprites using software). The Apple II version starts at 8:25.

I remembered that a few years ago Atari released, as open source, the code for a number of games for their 6502-based Atari 7800 system. I just found the dump on github:
https://github.com/OpenSourcedGames/Ata ... r/ROBOTRON
. In the Let's Compare video, this version can be seen at 13:49.

I didn't find the multiply subroutine that Chromatix analyzed, so it was likely an independent implementation. Still, it's an excellent, documented, reference for a real 6502 implementation of the game.



White Flame

Posts: 704
Joined: 24 Jul 2012

Re: reverse engineering Robotron 2084 for the Apple II

by **White Flame** » Thu Mar 07, 2019 5:20 am

Haven't had much time lately, and I'm too tired to do this (I say "16 byte" instead of "16 bit" too often), but I recorded a video anyway, fiddling around with some of the earlier Robotron routines in WFDis. The password is wfdis.

I decided to give vimeo a shot, since they're much less invasive than youtube, but unfortunately it doesn't have 1.5x or 2x playback speed, so you'll have to sit through it all at normal speed, which I can't stand doing anymore.

WFDis Interactive 6502 Disassembler
AcheronVM: A Reconfigurable 16-bit Virtual CPU for the 6502 Microprocessor



barrym95838

Posts: 2061
Joined: 30 Jun 2013
Location: Sacramento, CA, USA

Re: reverse engineering Robotron 2084 for the Apple II

by **barrym95838** » Thu Mar 07, 2019 8:19 am

Thank you so much for that very informative demonstration, WF! One question regarding your stack picture:



It looks like your simulation is inferring that $013d and $013e are being altered by the lda pha lda pha at the end of the copypage subroutine, but as I understand it, the 65xx post-decrements S for pushes and pre-increments S for pulls, causing $013e and $013f to be altered when given an initial S of $3f before the jsr.

Got a kilobyte lying fallow in your 65xx's memory map? Sprinkle some ***VTL02C*** on it and see how it grows on you!

Mike B. ***(about me)*** ***(learning how to github)***



White Flame

Posts: 704
Joined: 24 Jul 2012

Re: reverse engineering Robotron 2084 for the Apple II

by **White Flame** » Thu Mar 07, 2019 3:38 pm

Yeah, the emulator is really primitive and probably has some edge case bugs in it. Usually the stack effects while running don't need to be recorded in the output, so it hasn't been visible before. This is a weird case of overlapping features, with both JSR/RTS and STAs overwriting each other. But it still works well enough to figure out what goes on there.

WFDis Interactive 6502 Disassembler
AcheronVM: A Reconfigurable 16-bit Virtual CPU for the 6502 Microprocessor



fschuhi

Posts: 38

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Fri Mar 08, 2019 8:08 am

> **White Flame wrote:**
>
> Haven't had much time lately, and I'm too tired to do this (I say "16 byte" instead of "16 bit" too often), but I recorded a video anyway, fiddling around with some of the earlier Robotron routines in WFDis.

Joined: 23 Feb 2019

Thank you for putting in the time, that was a great intro into how to work with your tool. Very much appreciated.

The adhoc emulation with Shift-R is awesome. I didn't know that WFDis can do that. With regard to the task at hand, it was helpful to see how a subroutine address on the stack can be used to access data immediately after the JSR. That's an important idiom to know.

Prepending labels with L and S is sensible, so I have added this right away to my workbench. I would have expected that jumps were labeled with 'J', but you stick to 'L' here as well. Any particular reason why?

I had originally lowercased mnemonics and addresses (like seen for x86). Most of the snippets on this forum are uppercase, though. In WFDis you decided to lowercase everything. What is your reasoning behind deviating from the common "style guide"?)

The smooth scrolling is a neat idea. I can totally understand why this can help to understand how the different parts of the code hang together when there is much jumping around (as it is with Robotron). I lack the skill of keeping the structure of even smaller stretches of code in mind. For this reason I have put most of the work so far into automated structural analysis of the code. I do this in Python, because the emulator is written in Python, too. BTW Excel doesn't play a big role in the project, for me it's just a convenient notebook with additional intelligence and easy interface to the emulator and tracer.

I think using a manual disassembler is compatible with my reverse engineering approach. I lean on the ideas in Don Lancaster's "Tearing Into Machine-Language Code". He advises against using tools to disentangle the code and rather advocates "Do the dull stuff yourself!". But his method also assumes that those who want to reverse engineer should be versed in 6502. A bit more automated structure discovery is necessary to help my learning.

I'm going to continue working on the analysis over the weekend, let's see if I am able to share interim results. I would certainly consider transferring at least some of the automatically generated info to the listing in WFDis (e.g. caller/callee), so it would be nice if your next version re-enables inline comments. But that's just a nice to have, the

tool is certainly powerful as it is. Thanks again for the help!

ATTACHMENTS

Tearing Into Machine-Language Code (Lancaster 84).pdf
(180.99 KiB)



GARTHWILSON

Posts: 8822
Joined: 30 Aug 2002
Location: Southern California
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **GARTHWILSON** » Fri Mar 08, 2019 9:43 am

> **fschuhi wrote:**
>
> > **White Flame wrote:**
> >
> > it was helpful to see how a subroutine address on the stack can be used to access data immediately after the JSR. That's an important idiom to know.

That, and lots more, is in the 6502 stacks treatise which I called "6502 STACKS: More than you thought," this part being in section 7, "Inlining subroutine data."

http://WilsonMinesCo.com/ lots of 6502 resources
The "second front page" is http://wilsonminesco.com/links.html .
What's an additional VIA among friends, anyhow?



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

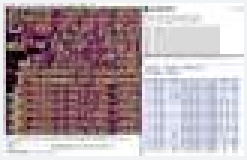
by **fschuhi** » Fri Mar 08, 2019 10:24 am

> **GARTHWILSON wrote:**
>
> > **fschuhi wrote:**
> >
> > > **White Flame wrote:**
> > >
> > > it was helpful to see how a subroutine address on the stack can be used to access data immediately after the JSR. That's an important idiom to know.

That, and lots more, is in the 6502 stacks treatise

> which I called "6502 STACKS: More than you thought," this part being in section 7, "Inlining subroutine data."

I had already found your treatise a couple of weeks ago! -- Very well explained and thus easy to understand, even for newcomers. In section 11 you say: "JSR and RTS are jumps with special effects on the hardware stack." When I read that it clicked.



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Fri Mar 08, 2019 10:44 am

> **fschuhi wrote:**
>
> I lean on the ideas in Don Lancaster's "Tearing Into Machine-Language Code".

That's an amusing read, thanks!

> **Quote:**
>
> You should have a tractor-feed printer along with some heavy white paper, preferably 20-pound. Naturally, you will also need a plastic 6502 Programming Card and, of course, the 6502 Programming Manual.

Post Reply 98 posts 1 2 3 4 5 ... 7

Return to "Nostalgia" Jump to

Forum Home Search Contact

All times are UTC

reverse engineering Robotron 2084 for the Apple II



White Flame

Posts: 704
Joined: 24 Jul 2012

Re: reverse engineering Robotron 2084 for the Apple II

by **White Flame** » Fri Mar 08, 2019 10:47 pm

> **fschuhi wrote:**
>
> Thank you for putting in the time, that was a great intro into how to work with your tool. Very much appreciated.

I need to work more on, and document more about, WFDis. Any use of it automatically gives me a little kick to do so for its own purpose as well.

> **Quote:**
>
> The adhoc emulation with Shift-R is awesome. I didn't know that WFDis can do that. With regard to the task at hand, it was helpful to see how a subroutine address on the stack can be used to access data immediately after the JSR. That's an important idiom to know.

Yes. When it comes to either pre-run snapshots (static analysis) or post-run snapshots (emulation), it's important to understand there is a flow of behavior beyond just the byte dump you happen to be seeing at one time. There might have been intermediate states in the memory footprint before/after, and that requires knowing what the code is actually doing, and why. Just looking at 1 state of memory might not reveal everything.

> **Quote:**

> Prepending labels with L and S is sensible, so I have added this right away to my workbench. I would have expected that jumps were labeled with 'J', but you stick to 'L' here as well. Any particular reason why?

No particular reason. "L" was the default, then I added "S" (I used "Sub" originally, but ended up being annoyingly long on earlier pre-html displays). I agree that a prefix for subroutine, branch/jump code, and data accesses would be reasonable.

> **Quote:**
>
> I had originally lowercased mnemonics and addresses (like seen for x86). Most of the snippets on this forum are uppercase, though. In WFDis you decided to lowercase everything. What is your reasoning behind deviating from the common "style guide"?)

Old code written way back in the 1000s was often on platforms with only uppercase available, or only uppercase by default, so there was little choice, and it created an ad-hoc standard. For most actual modern-written code bases, I believe most of them use lowercase. However, when posting code snippets and stating things inline in text (like here talking about LDA label,Y), it visually helps to distinguish the code from normal labels & prose by using uppercase, as ye olde code did. Automated output sometimes keeps uppercase to distinguish it from human-written code.

There've also been studies back in the day of single-case text that lowercase is easier to read than upper, but was determined too disrespectful for certain names and titles which should be Proper Cased, so they went with upper-only. I tend to find it more readable in lowercase as well, when looking at full listings.

> **Quote:**
>
> I lack the skill of keeping the structure of even smaller stretches of code in mind.

In order to get any level of this type of work done, you need to be able to just look at a few instructions at a time to see what they do, independent of the code that surrounds it. Each piece performs some deterministic mechanical step, and doesn't necessarily require the full structure to understand. The small basic blocks that I split up in the video are only like 2-4 instructions long, and are

pretty representative of what you need to determine the lowest granularity of documentable functionality. At that point, you should start looking at those small blocks as whole units that interact with each other, instead of always at the instruction level.

> **Quote:**
>
> For this reason I have put most of the work so far into automated structural analysis of the code. I do this in Python, because the emulator is written in Python, too. BTW Excel doesn't play a big role in the project, for me it's just a convenient notebook with additional intelligence and easy interface to the emulator and tracer.

I so far find structural analysis to be a bit of a red herring with 8-bit code. There is no ABI or anything that code needs to follow, except maybe calls to ROM routines. The control flow especially in games is convoluted for speed, not clarity, and most code takes shortcuts to make it easier to write as well. Timer or video-based interrupts can do weird mutations to code and data, because they need to be fast, and obviously don't fit in clean control flow graphs. Since there are so few registers, lots of transient state is passed around in memory, reusing memory locations for multiple purposes at multiple times.

So it doesn't make a lot of sense to me to try to map all of this hand-wrangled bit banging into some clean single model of execution. But that also depends on what you mean by "structural analysis".

> **Quote:**
>
> I think using a manual disassembler is compatible with my reverse engineering approach. I lean on the ideas in Don Lancaster's "Tearing Into Machine-Language Code". He advises against using tools to disentangle the code and rather advocates "Do the dull stuff yourself!". But his method also assumes that those who want to reverse engineer should be versed in 6502. A bit more automated structure discovery is necessary to help my learning.

As I mentioned above, I don't believe that the code/data separation is a significant portion of the work. Having static tracing or emulation traces really assists in those steps, though not without their faults (both can miss code

areas, for instance), but it's relatively easy to deduce which portions of the data are code:

- "Holes" of uncalled data surrounded by large code areas are often also code.
- A9 is "LDA #xx" which very commonly starts code paths.
- In my video example where one of the pointers went to an area with something like "00 00 10 20 30 ff ff ff" I assumed that it was not code, because it appears more structured like bitwise data.

With static tracing from some starting point (or emulation traces), it saves a ton of *tedium*, but the majority of *time* is still spent trying to glean understanding from the code, not the actual code/data separation.

What I find the most useful by far is giving names to things, especially variables and subroutines. Even when they're just guesstimates, once you name something and look at its various uses you can piece together a picture of what it's for, in a somewhat bottom-up fashion, but you need to focus on things that will reveal the most. There's a few good starting points to look at that can anchor some of this understanding:

- Accesses to well-known I/O addresses (keyboard/joystick inputs, video registers)
- Accesses to screen memory
- Functions that are called from many places (usually indicates main loop or small utility functions)
- Writes to known system or software vectors (which will generally point to code)
- Calls into ROM
- Initialization code can reveal the overall memory layout

These can help to name some of the primitives that the program is constructed from, which can start to make the overall structure more visible/readable.

Quote:

I'm going to continue working on the analysis over the weekend, let's see if I am able to share interim results. I would certainly consider transferring at least some of the automatically generated info to the listing in WFDis (e.g. caller/callee), so it would be nice if your next version re-enables inline comments. But that's just a

> nice to have, the tool is certainly powerful as it is.
> Thanks again for the help!

It can import label files, but not currently comments. The internal rewrite I'm still working on shouldn't change the current appearance or functionality much, but is necessary for future expansion and things like standalone and multi-line comments (right now all comments are tied to a single line of code as well). Cross-reference data is easily stored, but still remains on the TODO list for exposing graphically.

Last edited by White Flame on Fri Mar 08, 2019 11:41 pm, edited 6 times in total.

WFDis Interactive 6502 Disassembler
AcheronVM: A Reconfigurable 16-bit Virtual CPU for the 6502 Microprocessor



GARTHWILSON

Posts: 8822
Joined: 30 Aug 2002
Location: Southern California
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **GARTHWILSON** » Fri Mar 08, 2019 11:09 pm

> **White Flame wrote:**
>
> > **fschuhi wrote:**
> >
> > I had originally lowercased mnemonics and addresses (like seen for x86). Most of the snippets on this forum are uppercase, though. In WFDis you decided to lowercase everything. What is your reasoning behind deviating from the common "style guide"?)
>
> Old code written way back in the 1000s was often on platforms with only uppercase available, or only uppercase by default, so there was little choice, and it created an ad-hoc standard. For most actual modern-written code bases, I believe most of them use lowercase. However, when posting code snippets and stating things inline in text (like here talking about LDA label,Y), it visually helps to distinguish the code from normal labels & prose by using uppercase, as ye olde code did. Automated output sometimes keeps uppercase to distinguish it from human-written code.
>
> There've also been studies back in the day of single-case text that lowercase is easier to read than upper, but was determined too disrespectful for certain names and titles which should be Proper Cased, so they went

> with upper-only. I tend to find it more readable in lowercase as well, when looking at full listings.

Slightly O.T.; but since it came up, I'll add this for those who don't have habits firmly formed yet.

In prose, the lines on the page exist because a book is more manageable than a mile-long ribbon with a lump here and there for pictures. In programming however, the separation into lines becomes very significant for visual factoring, and having ascenders and descenders (as lower-case does) blurs those divisions. Similarly, numerals 0-9 are always "capitals." Do not mix lower-case a-f into them in hEx nUmBErS! Write for example $3EA9, not $3ea9. Here on the forum where the non-code sections are in proportional spacing, when someone writes for example 3fff, in whatever font is used here, it initially looks like 3111 to me because the f's are so narrow. (Actually now in the preview, they're showing up even narrower than 1's, which is backwards.) It's no harder to type capitals with the caps lock on. (I do take it off for comments.) If you feel like the computer is yelling at you, then turn the font size down. Easy enough. People who look through my code have said they like my style.

---

http://WilsonMinesCo.com/ lots of 6502 resources
The "second front page" is http://wilsonminesco.com/links.html .
What's an additional VIA among friends, anyhow?



White Flame

Posts: 704
Joined: 24 Jul 2012

Re: reverse engineering Robotron 2084 for the Apple II

by **White Flame** » Fri Mar 08, 2019 11:21 pm

> **GARTHWILSON wrote:**
>
> Here on the forum where the non-code sections are in proportional spacing, when someone writes for example 3fff, in whatever font is used here, it initially looks like 3111 to me because the f's are so narrow. (Actually now in the preview, they're showing up even narrower than 1's, which is backwards.)

In most proportional fonts, the digits are still uniform width, so a 1's character cell is still as wide as a 0's. Plus, kerning can mush 'f's even closer together. However, a lot of these textual display issues are why I chose HTML output to make it render very cleanly and legibly compared

to most bitmap font environments that other disassemblers stick with, even if it means a quite large memory footprint.

(Also, I do know people have complained about the colors before, which were copied over from a black-background prior version. Those colors will be darkened in the future as well, for better contrast in the white-background version.)

WFDis Interactive 6502 Disassembler
AcheronVM: A Reconfigurable 16-bit Virtual CPU for the 6502 Microprocessor



Chromatix

Posts: 1462
Joined: 21 May 2018

Re: reverse engineering Robotron 2084 for the Apple II

by **Chromatix** » Sat Mar 09, 2019 12:25 am

I actually spent some time last week making a proportional bitmap font - in which the uppercase A-F are purposely the same, constant width as the digits 0-9 (and thus slightly wider than most other capitals).



sark02

Posts: 241
Joined: 10 Nov 2015

Re: reverse engineering Robotron 2084 for the Apple II

by **sark02** » Sun Mar 10, 2019 12:31 am

> **GARTHWILSON wrote:**
> Do not mix lower-case a-f into them in hEx nUmBErS! Write for example $3EA9, not $3ea9.

To provide an opposing view: I prefer lower case hex over upper case, as it's easier for me to differentiate e and f (vs. E and F).

There's no wrong answer, just personal preference.

I learned all my 8-bit assembly (Z80, 6502, 6809) exclusively upper-case, but everything since has been lower-case.



GARTHWILSON

Posts: 8822
Joined: 30 Aug 2002
Location: Southern California
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **GARTHWILSON** » Sun Mar 10, 2019 12:39 am

> **sark02 wrote:**
>
> I prefer lower case hex over upper case, as it's easier for me to differentiate e and f (vs. E and F).

Then how about a versus e, as in $3EA9 and $3ea9? I think personal preference is your key.

---

http://WilsonMinesCo.com/ lots of 6502 resources
The "second front page" is http://wilsonminesco.com/links.html .
What's an additional VIA among friends, anyhow?



barrym95838

Posts: 2061
Joined: 30 Jun 2013
Location: Sacramento, CA, USA

Re: reverse engineering Robotron 2084 for the Apple II

by **barrym95838** » Sun Mar 10, 2019 4:05 am

> **GARTHWILSON wrote:**
>
> > **sark02 wrote:**
> >
> > I prefer lower case hex over upper case, as it's easier for me to differentiate e and f (vs. E and F).
>
> Then how about a versus e, as in $3EA9 and $3ea9? I think personal preference is your key.

Those are all equally readable to me, as long as they're in a fixed-width font. I don't try to impose my preferences on others, and kind of "go with the flow". If I'm replying to a post with suggestions of my own, I usually make a cursory effort to match the style of the original source. If I'm coding for myself, I almost always use lower case if it's available, and I have recently switched exclusively to spaces instead of tabs for my own creations, but that's just how I roll.

Got a kilobyte lying fallow in your 65xx's memory map? Sprinkle some ***VTL02C*** on it and see how it grows on you!

Mike B. ***(about me)*** ***(learning how to github)***



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Sun Mar 10, 2019 11:57 am

> **White Flame wrote:**
>
> > **Quote:**
> >
> > I lack the skill of keeping the structure of even smaller stretches of code in mind.
>
> In order to get any level of this type of work done, you need to be able to just look at a few instructions at a time to see what they do, independent of the code that surrounds it. Each piece performs some deterministic mechanical step, and doesn't necessarily require the full structure to understand. The small basic blocks that I split up in the video are only like 2-4 instructions long, and are pretty representative of what you need to determine the lowest granularity of documentable functionality. At that point, you should start looking at those small blocks as whole units that interact with each other, instead of always at the instruction level.

This seems to be a common theme: think in idioms or building blocks which interact with each other. 6502 with its small number of statements takes up much vertical space in the disassembler listing. Scrolling takes time and destroys the immediate context. In order to trace what's happening mentally, one must therefore be able to see those small repeating patterns standing out.

My statement was meant a little differently, though. I still focus on just the first seconds of the intro. That's just 500 instructions which are of course not consecutive in memory but organised in "stretches" of different length in various memory locations. For the purpose of reverse engineering, even the best tools cannot replace the ability to combine those stretches into some form of spatio-temporal mental model of the code.

> **Quote:**
>
> So it doesn't make a lot of sense to me to try to map all of this hand-wrangled bit banging into some clean single model of execution. But that also depends on what you mean by "structural analysis".

"Structural analysis" is that part of the static analysis of the code that discovers how parts of the code hang together, at different levels of granularity. The success depends a bit on how "rational" the code is structured.

Most of the machine code reverse engineering nowadays is probably happening in the anti-virus space. I suspect that at least some of the virus code out there is heavily obfuscated, on top of the usual bit banging which is part of the territory when dealing with machine language. Still, specialists regularly break into that virus code uses a combination of a number of tools, using a combination of static and dynamic analysis.

It should be possible to attack an 8-bit game like Robotron in a comparable fashion. Most of them were handcoded by single developers who needed to keep a mental model of the whole thing in their memory. In this case that was originally Eugene Jarvis, a developer known for rational internal design of his games. Comparing the Williams Arcade and the Atari versions, they are different but seem to implement a comparable execution flow. I suspect that Steve Hays, who ported the game to the Apple II, used at least some of the original ideas. It should therefore be possible to derive quite a bit of what I call "structure" automatically.

Only a bit of the work can be automated, of course, but some of it can. If I've understood you correctly that's also what's happening on the server version of WFDis - - some automated discovery which complements the always needed interactive work with the code.

> **Quote:**
>
> a few good starting points to look at that can anchor some of this understanding:

My analysis is helped by the fact the the Apple II is a very simple system (e.g. no interrupts). The non-intuitive parts (especially hires graphics) are well documented. It's thus easy to know which parts of the code are responsible for graphics output. Another easy part of the Apple II are the softswitches for keyboard access and game controller input. I've already added the relevant softswitch addresses as labels. And Robotron doesn't reload anything from disk, so it's benign even from this point of view.

All the automated structure discovery is only necessary but

far from sufficient for understanding what and especially why the code does what it does. I really like the 6502, though, so I will eventually manage to peek behind the one or the other curtain

Last edited by fschuhi on Sun Mar 10, 2019 12:22 pm, edited 1 time in total.



fschuhi

Posts: 38
Joined: 23 Feb 2019

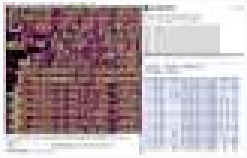
Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Sun Mar 10, 2019 12:12 pm

I knew before that the people in this forum are really both knowledgeable and helpful, but to experience it first-hand is something different. - -

Many thanks to all of you who have contributed to this topic so far. I feel I have been making soo much more progress than if I had fiddled with the task on my own. Your posts cover a lot of breath and depth, they are really a great resource.

Congrats for having built such a community



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Sun Mar 10, 2019 5:41 pm

Very nice to hear those thoughts Frank, thanks for expressing them! You're on a great journey, please keep up with questions and observations and progress reports.

(BTW I think perhaps the 'basic block' might be found in any code, which is to say straight-line sections in between entry points, branches, and returns. Backward branches are always good to see because they are often loops, and studying the setup and loop body can be quite illuminating.)



fschuhi

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Mon Mar 11, 2019 10:17 am

Attached an Excel workbook with a disassembly listing ("asm") built from tracing the execution of a couple of seconds of the Robotron intro. The emulator tracks the leaps from and to instructions, then runs an algorithm to structure the code logically. I've implemented a number of suggestions I got from above posts, thanks for the

Posts: 38
Joined: 23 Feb 2019

inspiration!

As a complement to "asm" I've added "map" (condensed picture of the memory of the code) and "ann" (annotations to addresses). For reference purposes I have added the complete listing on "raw" (generated by a static disassembler). Not included in the workbook are the Excel macros which drive the workbench, so it's just a snapshot.

Some explanations to make reading the listing on sheet "asm" easier:

- The addresses shown are consecutive in memory. This leaves big swaths of .byte data. Some of the data can be identified as ascii. These are further screens of the intro which have not been touched in this particular run. I intend to approach those further parts of the code in a piecemeal fashion, always trying to identify some structures before moving on with discovering of new stretches of code.
- Most of the labels are auto-generated (S for JSR entry points, L for branches, J for JMP targets).
- There are some manual labels which are supplied to the emulator before it starts, e.g. chromatix01 (multiplication routine as pointed out by chromatix) or copyPagesAtoY (explained in White Flame's video).
- Some special labels show access to Apple II softswitches and ROM routines.
- The comments contain a lot of '>' and '<' info. That's the addresses this particular instruction leaps to (>), or where other instructions leap from into this instruction (<).
- Branch instructions can always branch (+), never (-) or sometimes (0). Branching always can create holes behind the branch instruction if we never get there during execution.
- Blank lines are added automatically, at "sensible" points in the listing (as determined by the "tiles" of code, explained below).
- "touch" shows the number of times an instruction was executed. This helps me to identify the small-scale structure because it makes loop stand out with higher touch counts. It is also neat to identify initialisation code which is only executed once.
- "first" and "last" are cycle counts. The emulator does not store every step of the execution path (which

would be excessive) but rather builds a static map from its action.

Apart from the labels and leap targets in the comments, the main work in terms of automatically structuring the code is done with combining vertical parts of the code into bigger logical entities:

- Instructions are combined to *"tiles"*. Conceptually, the listing is cut immediately behind a leap (JSR, RTS, JMP, Bxx) and immediately before the address where leaps enter. As a result, all small loops have their body and the Bxx on the same tile. Because tiles break the stream of instructions at points where the code leaps in or out, a tile is a local path of sequential execution. Tiles are indicated by a dotted gray line above their first instruction.
- Tiles can be combined into *"stretches"*. For example, a tile with a JSR at the end can be connected to the following tile if the latter has as its only entry point a leap back from the matching RTS. The execution path also traverses over tile boundary if a branch is not taken, but those tiles can be connected, too. There are a handful of rules which guide how to connect tiles to grow stretches.
- Some stretches are *"compact"*: They start w/ a tile only leaped to by (or or more) JSR, contain no jumps and only JSR to other stretches which return normally (i.e. normal stack behaviour). Such a compact stretch is marked with a solid red line in the listing, other stretches with a dotted red line.

Compactness is currently defined in a way that it is possible for a stretch to be compact even if it JSRs to a non-compact stretch. Still, compact stretches are very good candidates for regular subroutines. I am well aware that my definition of compactness has a serious flaw: A stretch can be marked as compact even if there are other stretches which leap into other tiles than the head of the stretch, or if grandchildren in the call order are non-compact. I will add this deeper stretch info as soon as I have an idea how to present it on the Excel workbench.

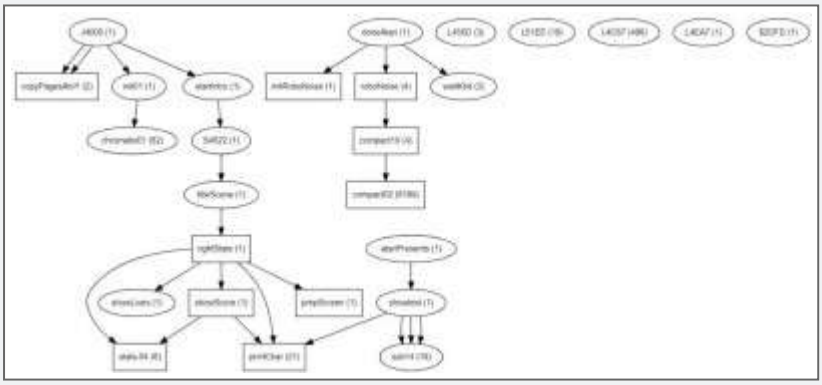
I currently need a lot of time to understand even superficially what a compact stretch does, mechanically. In order to speed up the understanding from a bird's eye perspective, though, I can of course just switch off the execution of a stretch by poking an RTS at the entry point. Repeatedly re-running the software in the emulator then

shows which stretch does what. It took just a couple of minutes to identify the routines which print the stats (score and lives) on the right side of the screen, even though I don't know how the 6502 code achieves that.

In this way I am able to make progress and continuously engage with the 6502 machine language by looking at the stretches and the smaller tiles. As long as there is no way to add comments or blank lines manually, the tool is not useful for that kind of reverse engineering anyways. I haven't made up my mind yet how to implement it because I want to focus on the discovery of broader structures anyways, at least for now.

Visually, there is still a bit too much happening in the listing now. I will probably remove the tile separators, because the blank lines already do the trick. A lot of the leap to/from info in the comments will turn out to be not helpful for understanding the code. That's fine, I can always reduce the information when I start to "see" the branch patterns and idioms.

The list of stretches can be shown as a call graph (generated automatically):



Remarks:

- The address or label shown is the address of the first instruction in the first tile of the stretch.
- Compact stretches are shown as boxes, non-compact ones as ellipses. Some of the stretches are floating disconnected around. That's a nice TODO list for further refactoring, because it shows where the algorithm is not yet able to connect enough tiles to form meaningful stretches.
- The arrow is not the call itself but rather pointing from one stretch to another one. Any JSR from inside any of the tiles of a stretch will be shown as just an outbound arrow and any leaps into the body of a stretch would show as an inbound arrow.

- Consequently, the call graph is a condensed picture of the JSR action happening between the stretches.
- The call graph does not show JMP yet.
- I detect RTS jumps which are not balanced by a JSR. The RTS at $51CA is such a "RTS*" because we return to an address after the stash which contains the text to be printed, as explained above for atariPresents and doneAtari. The instruction is marked as RTS* on the "map" sheet, but I don't do anything yet with it.

With the total freedom one has over the stack and execution flow in general, the automated analysis will always fail in certain spots. Currently something like atariPresents is a degenerated stretch: just one tile with one JSR to showText, because the text to show is in the stash below the JSR. Knowing that this is a well-established idiom, the algorithm should be able to build a bridge over atariPresents / stash / doneAtari. This would

- reduce the visual cluttering by eliminating the red dotted lines,
- make the listing more organised with more solid red lines, and
- would also help to cut down the call tree.

Furthermore, sometimes the algorithm tries to be too intelligent. For example, a part of the stretch detection algorithm is to connect tiles ending in an always-branch (+) with the following tile. The assumption behind this is that the branch instruction would fall through eventually. This is usually true, of course, but an SEC/BCS combination (twice in the code) does really never fall through by definition. This shows how progress can be made with the emulator:

- identify a weird situation in the listing or call tree
- help the algorithm manually to do the right thing
- check the newly grouped listing and the call tree
- implement the rule as part of the automated rule set

For other occurances the algorithm then already knows how to interprete the situation correctly, thus showing correct stretches and making it easier for me to understand what's happening.

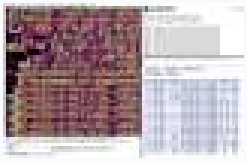
The algorithm will change anyways as soon as I introduce post-execution disassembling, i.e. deviating from the rule "only use executed instructions". This will surface lone RTS

like the one in $4EA6 and solve the problem of important parts of loop structures (i.e. the INC in $4C65, as pointed out by chromatix). There are also a number of '-' Bxx which never branch but obviously do at least sometimes. It is not possible to get complete coverage without recursively traversing code. That's a bigger change in the algorithm, though, I will have to plan a bit how to do it before changing the current algorithm.

A huge helper for understanding the 6502 details will be fine-grained LD/ST tracking. The issue is less how to do it in the emulator but rather how to constrain it. One of the reasons for this is that Python is simply too slow for doing this kind of on the fly data collection. But the main reason is that the amount of data generated from recording all LD/ST is massive which would actually hurt the understanding. There are certainly many design choices how to approach this conundrum and find a good balance - - any ideas are welcome.

ATTACHMENTS

Robotron 090311.xlsx
(702.58 KiB)



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Mon Mar 11, 2019 10:45 am

If there's an easy way for you to print to PDF and attach that, it would be helpful!

> **fschuhi wrote:**
>
> In order to speed up the understanding from a bird's eye perspective, though, I can of course just switch off the execution of a stretch by poking an RTS at the entry point. Repeatedly re-running the software in the emulator then shows which stretch does what. It took just a couple of minutes to identify the routines which print the stats (score and lives) on the right side of the screen, even though I don't know how the 6502 code achieves that.

That's a nice idea! The opposite of instrumentation, removing code.

> **Quote:**
>
> For example, a part of the stretch detection algorithm is to connect tiles ending in an always-branch (+) with

> the following tile. The assumption behind this is that the branch instruction would fall through eventually. This is usually true, of course, but an SEC/BCS combination (twice in the code) does really never fall through by definition.

Might be worth pointing out that skilled 6502 programmers will often be able to make an always-branch by using higher-order understanding of the preceding code. For example, knowing that something is never zero, or never negative, or that carry will always (or will never) be set by some preceding operation.

It's probably clear to you by now, but one aspect of the 6502 flags is that they are not updated by all instructions. Therefore, a branch being taken or not taken can be a consequence of something which happened several instructions earlier, possibly prior to a branch or even a call. So, there are four bits of state, in effect, which kind of act like short-lived variables. The Z flag is relatively often affected, the V flag relatively rarely. When the flag is affected prior to an RTS and then affects control flow in the parent routine, it's being used as a return code - but that's just one example. If you ever see PHP and PLP then of course that makes the flag values persist for much longer - perhaps around a complex section of code, or even around some subroutine calls.



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Mon Mar 11, 2019 11:16 am

BigEd, I've saved the "asm" sheet as PDF to retain the colors, the other sheets as csv.

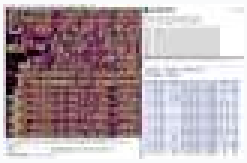
ATTACHMENTS

raw 090311.csv
(285.6 KiB)

ann 090311.csv
(19.06 KiB)

map 090311.csv
(26.54 KiB)

asm 090311.pdf
(684.52 KiB)



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Mon Mar 11, 2019 11:55 am

Thanks! I notice very opcode-like byte values in the stretches of as-yet-undisassembled code. It might be worthwhile to apply some statistical test to see whether each block looks like code, or ascii, or other.



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Mon Mar 11, 2019 12:14 pm

> **BigEd wrote:**
>
> Thanks! I notice very opcode-like byte values in the stretches of as-yet-undisassembled code. It might be worthwhile to apply some statistical test to see whether each block looks like code, or ascii, or other.

Yes, most of the .byte sections are code, which is also evident from the disassembler listing in the "raw" csv. That output was generated by a static disassembler, starting at $2DFD, the load address of the binary.

For a quick statistical test to check if a block of bytes s code or not I would just point the static disassembler to the first byte of the block, disassemble until the last byte and then check for any unknown ops. Optional checks might be multiple BRK or ORA, or any repetition of bytes >4 times. Are there any standard approaches?

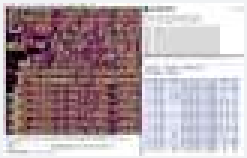
Post Reply 98 posts 1 2 3 4 5 6 7

Return to "Nostalgia" Jump to

Forum Home Search Contact

All times are UTC

reverse engineering Robotron 2084 for the Apple II



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Mon Mar 11, 2019 12:37 pm

It's a good point – if you have a disassembler already, then that's a way to check if a block is code or not. (You might need to be a little fuzzy – 90% good disassembly means it's code, for example.) Nice idea on spotting too many BRKs. I'm not aware of any guidance on doing this.



White Flame

Posts: 704
Joined: 24 Jul 2012

Re: reverse engineering Robotron 2084 for the Apple II

by **White Flame** » Mon Mar 11, 2019 7:07 pm

> **fschuhi wrote:**
>
> This is usually true, of course, but an SEC/BCS combination (twice in the code) does really never fall through by definition.

As you're talking about a stretch, and not a tile in your specific vocabulary, the above is not technically true. If the BCS is a leap target, then even though a SEC immediately precedes it, it only does so in one code path and thus doesn't always branch.

> **Quote:**
>
> A huge helper for understanding the 6502 details will be fine-grained LD/ST tracking. The issue is less how to do it in the emulator but rather how to constrain it. One of the reasons for this is that Python is simply too slow for doing this kind of on the fly data collection.

> But the main reason is that the amount of data generated from recording all LD/ST is massive which would actually hurt the understanding. There are certainly many design choices how to approach this conundrum and find a good balance - - any ideas are welcome.

I do this in the server in my tools (btw, the server based one isn't in a running state, the TODO list is large and has many codependent items that need to come together), but LD/ST isn't the only way that things in the CPU state get set. Things like TAX are a write to .X, and INX both reads the old state of .X and writes a new one as well. It's also very useful to track status register flags in the same way. It's handy as a cross-reference list, and isn't that bad when displaying it as extra context off to the side of each instruction.

> **fschuhi wrote:**
>
> For a quick statistical test to check if a block of bytes s code or not I would just point the static disassembler to the first byte of the block, disassemble until the last byte and then check for any unknown ops. Optional checks might be multiple BRK or ORA, or any repetition of bytes >4 times. Are there any standard approaches?

That's probably reasonable. Any BRK or RTS should probably end the attempt, and then see if it's worth keeping. Personally, I think this is a reasonable place for a neural net, as it's literally a classification problem.

WFDis Interactive 6502 Disassembler
AcheronVM: A Reconfigurable 16-bit Virtual CPU for the 6502 Microprocessor



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Mon Mar 11, 2019 8:46 pm

> **White Flame wrote:**
>
> > **fschuhi wrote:**
> >
> > This is usually true, of course, but an SEC/BCS combination (twice in the code) does really never fall through by definition.

> As you're talking about a stretch, and not a tile in your specific vocabulary, the above is not technically true. If the BCS is a leap target, then even though a SEC immediately precedes it, it only does so in one code path and thus doesn't always branch.

Correct, I need to keep that in mind, the SEC needs to be on the same tile. Furthermore, as explained by BigEd: An SEC can be *anywhere* on the execution path before the BCS, making the BCS branch always. The situation cannot be resolved clearly.

> **Quote:**
>
> but LD/ST isn't the only way that things in the CPU state get set. Things like TAX are a write to .X, and INX both reads the old state of .X and writes a new one as well.

I had only the memory load/store in mind, but of course there is also the whole class of bit operations which operate on the memory. I can hook into the read/write of the simulated memory and collect the info. Even that can quickly lead to just too many references, especially with indexed access, so I need to condense it, maybe grouping by page.

> **Quote:**
>
> Any BRK or RTS should probably end the attempt, and then see if it's worth keeping.

Also RTI, because the Apple II doesn't work with interrupts (only for serial cards or mouse cards, nothing I have to care about here).

> **Quote:**
>
> Personally, I think this is a reasonable place for a neural net, as it's literally a classification problem.

My first reaction was "that would be an overkill"  but depending on the task it might actually be a good idea to train the nn on detecting the edges between code and non-code. Does your server use machine learning on the code?

For the task at hand I think I can get to good results by flood filling .byte sections from known entry points, e.g. branches immediately before (i.e. falling through into) the .byte block, or a branch into the .byte block which never

executed. The stop condition would then be an (unmatched, see below) RTS, because even newly disassembled absolute JMPs and JSRs could be treated as hints where to continue to disassemble. Indirect JMP needs to be treated as a stop condition, too, because it's not possible to resolve it statically.

You use flood-filling in your server, right? How do you treat JSR? Most of the JSR will return with a regular RTS, so spidering the subroutine(s, recursively) is sensible. On the other hand, some JSR return to a point after the params which follow immediately behind the JSR, e.g. the into stuff you disassembled in the video. I will probably start w/ JSR as stop condition for the spidering, then relax this with JSRs to stretches known to be compact.



Tor

Posts: 597
Joined: 10 Apr 2011
Location: Norway/Japan

Re: reverse engineering Robotron 2084 for the Apple II

by **Tor** » Tue Mar 12, 2019 9:01 am

For me, with my particular vision issues, $03ea9 is easier to read than $3EA9. Likewise $4abc vs $4ABC
The problem (for me) with the capital variant is that capital letters tend to lean more vertically up on the digits (e.g. $03E, vs $03e), and even letter-to-letter, so it's slightly harder for me to read. In other words, it's faster and more accurate for me to read lower case hex numbers. I don't do well with vertically close lines.



White Flame

Posts: 704
Joined: 24 Jul 2012

Re: reverse engineering Robotron 2084 for the Apple II

by **White Flame** » Tue Mar 12, 2019 9:21 am

> **fschuhi wrote:**
>
> My first reaction was "that would be an overkill" but depending on the task it might actually be a good idea to train the nn on detecting the edges between code and non-code. Does your server use machine learning on the code?

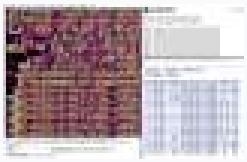
I haven't done anything with NNs looking at code bytes yet. It's obviously very platform-specific, too, and I'd want to be fully cross-platform.

> **Quote:**

> You use flood-filling in your server, right? How do you treat JSR?

The target for the server is strongly associated with symbolic execution. Every byte of code needs an impetus for being declared as code, rooted by the file format's entry points and the user's declaration of starting points. Automatically disassembling totally unused code in the file is something I'm not going to tackle soon; finding out how to find all the code paths (indirection, stack games, loaded overlays, interrupts, runtime codegen, self-modification) is what I want it to do instead.

WFDis Interactive 6502 Disassembler
AcheronVM: A Reconfigurable 16-bit Virtual CPU for the 6502 Microprocessor



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Tue Mar 12, 2019 9:30 am

It might be interesting to note that the usual unsophisticated approach is to run the disassembler over everything, and then to examine the failures, which might be code blocks or misaligned disassembly, and only eventually to distinguish any code which might be unreachable. (I would think unreachable code would be rather unlikely, and data which successfully disassembles also unlikely. But both could happen, especially on a fine-grained view.)



fschuhi

Posts: 38
Joined: 23 Feb 2019

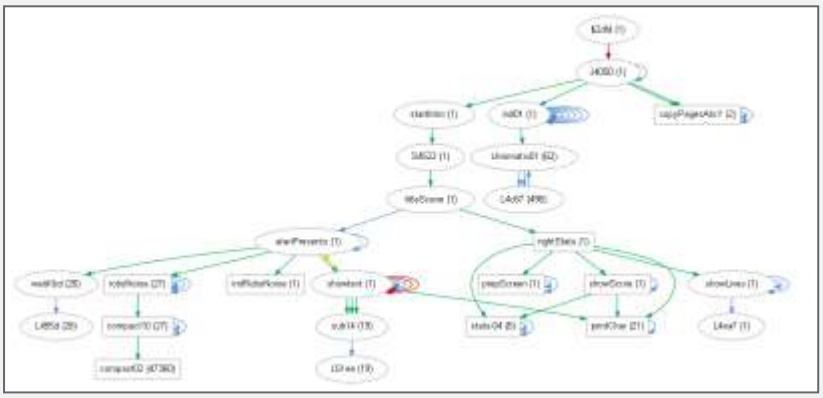
Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Sat Mar 16, 2019 8:02 pm

I've continued working on a way to generate the "big picture" of what's happening. As an example, here is the screen after a couple of seconds of execution:



This is the corresonding auto-generated call-tree.



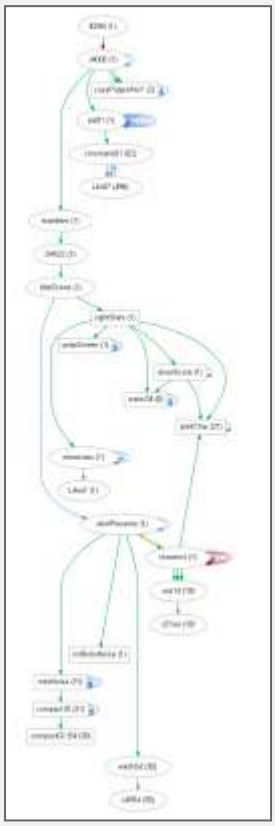
- JSR = green
- JMP = red
- branchings (if taken) = blue
- RTS if not paired with JSR

The paired RTS are not shown, if they did they would return to the sources of the green arrows. The *showgext* sub displays "Atari presents" at the top of the screen. The string is supplied as param immediately after the JSR to *showtext*, therefore the RTS does not match that JSR and is shown in yellow.

The nodes are "stretches" of code which the automatic analysis shows to be related. For the nodes with a box shape the analysis determines that it is a regular subroutine. Arrows shown are between stretches, not between the actually leaping instruction (which is somewhere in a stretch) and the target it leaps to (which is in the same or a another stretch). This pruning of the call tree is important, especially for bigger trees, because with otherwise the huge number of arrows and nodes gets confusing and unwieldly. As a side effect, complex stretches with many branches catch the eye (*init01*,

showtext with JMP-loops inside the stretch).

The call tree gives a good impression of the overall structure and gives hints on the execution flow. For small call trees that's fine, but for the bigger ones I am currently trying to understand it is difficult to discern what comes first. I've therefore added the following alternative view:



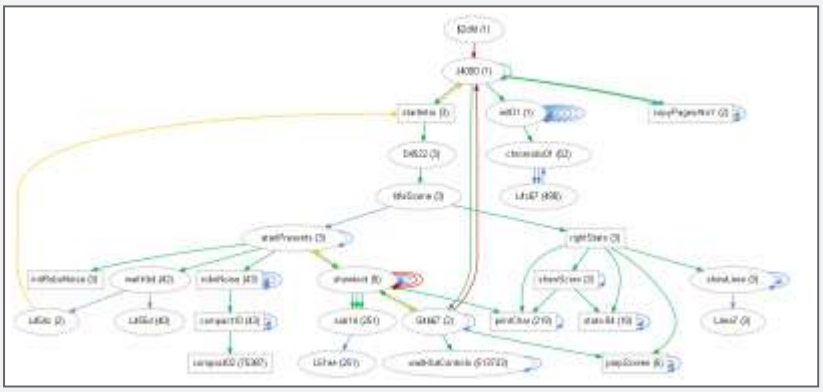
Here the nodes are ordered vertically by the cycle count of the first execution of that particular stretch. (The nodes are randomly ordered in horizontal direction, to fill the available space as best as possible). You can follow the *init01* doing its work in the beginning, then the intro starts with the title scene which contains a part which shows the stats on the right side of the screen, then the "Atari presents" and then the noisily dotted "Robotron" animation.

printChar is used twice: first to show the score, later to show the "Atari presents". A node is located where its stretch executed for the first time, so there is an upward JSR arrow from *showtext* to *printChar*. Showing nodes only once is another type of pruning which has turned out really helpful for bigger trees.

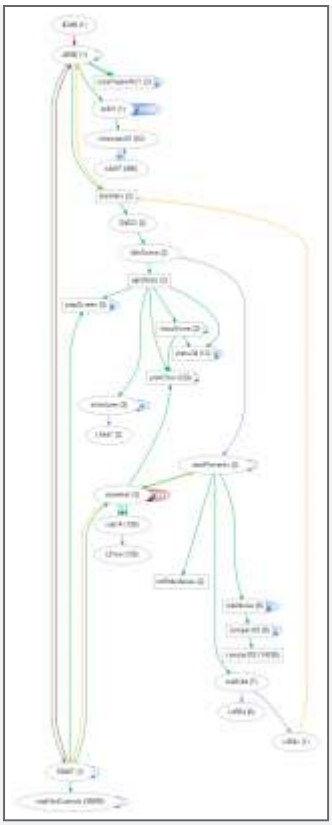
The second alternative also shows major loops. If I press space then the intro screen changes to the control selection:



If I press escape then the intro starts over with the "Atari presents" and the "Robotron" visualisation. The corresponding call trees:



The call tree shows the return to the intro sequence, but using the cycle-timed layout the flow of execution becomes more evident, with the yellow RTS jumps showing how the code is stitched together:



Together with the load/save snapshot feature of the workbench this has really given my understanding a good push forward. But it's of course just the superficial picture of the executing code. More insight will have to come from not only seeing the flow but also understanding which stretches read and manipulate memory. I haven't started with this topic yet, though.



White Flame

Posts: 704
Joined: 24 Jul 2012

Re: reverse engineering Robotron 2084 for the Apple II

by **White Flame** » Sat Mar 16, 2019 10:50 pm

> **fschuhi wrote:**
>
> Here the nodes are ordered vertically by the cycle count of the first execution of that particular stretch.

That's a great idea. I've personally never gotten much value out of call graphs, but this effectively turns it into a readable flowchart. Nice!

WFDis Interactive 6502 Disassembler
AcheronVM: A Reconfigurable 16-bit Virtual CPU for the 6502 Microprocessor

Re: reverse engineering Robotron 2084 for the Apple II

by **Chromatix** » Sun Mar 17, 2019 1:23 am



Chromatix

Posts: 1462
Joined: 21 May 2018

Now might be a good time to mention another idiom, not restricted to the 6502: a subroutine may be entered using JMP instead of using JSR immediately followed by RTS. This is known as a tail-call optimisation. On the 6502 it saves 1 byte and 6 cycles.



GARTHWILSON

Posts: 8822
Joined: 30 Aug 2002
Location: Southern California
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **GARTHWILSON** » Sun Mar 17, 2019 2:50 am

> **Chromatix wrote:**
>
> Now might be a good time to mention another idiom, not restricted to the 6502: a subroutine may be entered using JMP instead of using JSR immediately followed by RTS. This is known as a tail-call optimisation. On the 6502 it saves 1 byte and 6 cycles.

actually 9 cycles, since JMP is 3 cycles shorter than JSR (3 versus 6), and the saved RTS is another 6.

http://WilsonMinesCo.com/ lots of 6502 resources
The "second front page" is http://wilsonminesco.com/links.html .
What's an additional VIA among friends, anyhow?



fschuhi

Posts: 38
Joined: 23 Feb 2019

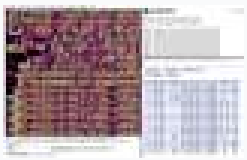
Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Sun Mar 17, 2019 10:55 am

> **GARTHWILSON wrote:**
>
> > **Chromatix wrote:**
> >
> > Now might be a good time to mention another idiom, not restricted to the 6502: a subroutine may be entered using JMP instead of using JSR immediately followed by RTS. This is known as a tail-call optimisation. On the 6502 it saves 1 byte and 6 cycles.
>
> actually 9 cycles, since JMP is 3 cycles shorter than JSR (3 versus 6), and the saved RTS is another 6.

That's a nice one, thanks for pointing that out. So far I've only encountered simple loops and "exceptions"

(PLA/PLA/JMP), but the main code is riddled with JMP,
there might be the one or the other tail call among them.



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Sun Mar 17, 2019 1:16 pm

This reminds me of another thing, or three...

- a routine can fall through into the routine that it would have tail-called, if it is positioned right afterwards. Just the same, but without the JMP.
- a routine which needs to be run twice, such as a nibble-processing routine, can be called immediately before falling through:

CODE: SELECT ALL

```
... some code
JSR dothething
dothething:
... the thing to be done twice
RTS
```

- and even again for something to be done four times (and I do believe I've seen this done, perhaps to deal with 4 bytes of a 32-bit value):

CODE: SELECT ALL

```
... some code
JSR dotwothings
dotwothings:
JSR dothething
dothething:
... the thing to be done twice or even four times
RTS
```



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Thu Mar 21, 2019 7:46 am

I've managed to understand some important subroutines completely, on my own, for the first time ever

(Don't worry, I won't bother you every time when I understand anything new, it's just a heads-up)

This is a snippet which is part of the game's intro screen:

CODE: SELECT ALL

```
$456c  atariPresents JSR showtext
                     ; $0d marks new line, followed b
                     ; x in chars (0-39), y in hires
$456f                .byte 0d 0c 12 "ATARI PRESENTS:"
$4582  doneAtari     RTS
```

showText is pretty straightforward:

CODE: SELECT ALL

```
$51b8  showtext      PLA               ; save return
$51b9                STA $22           ;   will be in
$51bb                PLA
$51bc                STA $23

$51be  J51be         JSR readNextChar  ; main loop, c
$51c1  R51c1         TAX
$51c2                BNE L51cb         ; is it #$00?

$51c4  L51c4         LDA $23           ; yes, we reac
$51c6                PHA               ; point stack
$51c7                LDA $22
$51c9                PHA
$51ca                RTS               ; RTS to addre

$51cb  L51cb         CMP #$0d          ; start new li
$51cd                BNE L51dc

$51cf  L51cf         JSR readNextChar  ; yes, load x-
$51d2  R51d2         STA $e0           ; $e0 = x (0-3
$51d4                JSR readNextChar  ; load y-posit
$51d7  R51d7         STA $e1           ; $e1 = y (0-1
$51d9                JMP J51be         ; start readin

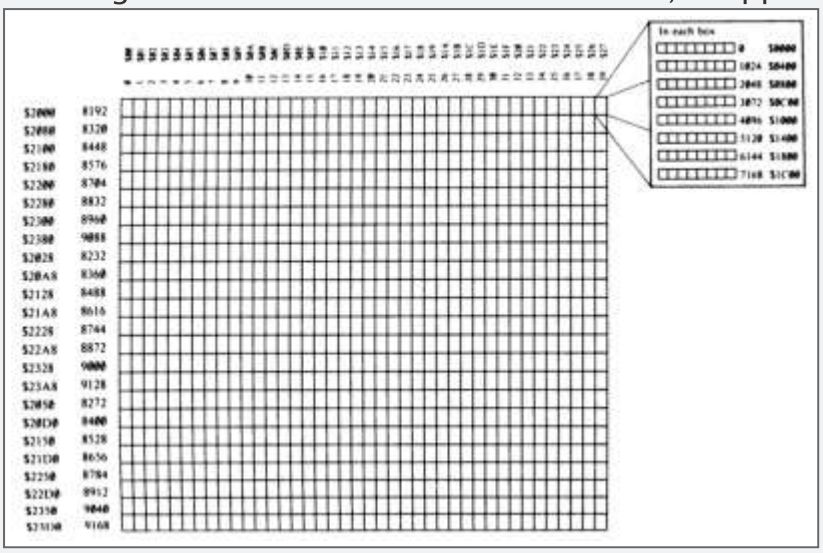
$51dc  L51dc         LDX $e0           ; read char is
$51de                LDY $e1           ;   => display
$51e0                JSR printChar
$51e3  R51e3         INC $e0           ; move output
$51e5                JMP J51be

$51e8  readNextChar  INC $22           ; next char in
$51ea                BNE L51ee
$51ec                INC $23           ; stash can be

$51ee  L51ee         LDY #$00
$51f0                LDA ($22),Y       ; load char fr
$51f2                RTS
```

Understanding the *printChar*, on the other hand, needed info about how the Apple II deals with hires graphics. The

way the addressing of pixels works was optimised for needing as little hardware as possible. Certainly ingenious at the time, but from today's vantage point it resides in the "Nostalgia" board of this forum for a reason, I suppose.



TL;DR: The layout of the pixels on screen differs substantially from the layout of bits in screen memory.

If I remember correctly, this was the point in high school where I stopped dealing with Apple II lowlevel stuff and branched off to Turbo Pascal etc. Today I have access to many excellent primers on the net, including PDFs of Apple II manuals, so "understanding on my own" doesn't really hit the mark, of course.

Looking at *printChar*, it's not difficult at all to deal with the arcance addressing scheme:

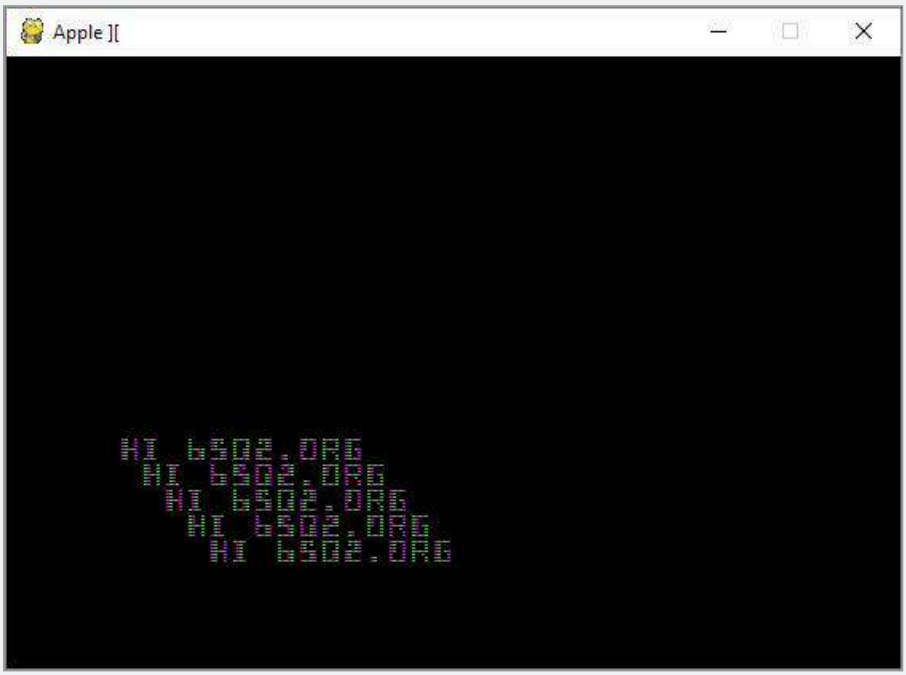
CODE: SELECT ALL

```
$5108  printChar     STY $fc           ; Y has hires
$510a                STY $fe           ;   ($fc) = ba
$510c                LDY #$40          ; will be ROLd
$510e                STY $09
$5110                SEC
$5111                SBC #$20          ; ascii table
$5113                ASL               ; A has char,
$5114                ASL               ; 3x ASL + ROL
$5115                ASL
$5116                ROL $09           ; ($08) has po
$5118                STA $08
$511a                LDY #$07          ; each char ha

$511c  L511c         LDA ($fc),Y       ; determine of
$511e                STA $5129
$5121                LDA ($fe),Y
$5123                STA $512a
$5126                LDA ($08),Y       ; load char pi
$5128                STA $2900,X
$512b                DEY
```

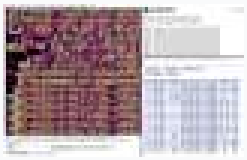
```
$512c           BPL L511c       BPL goes from
```

A neat feature is the self-modifying code in L511c. $fd contains $12 and $ff contains $13, so the tables in $1200 and $1300 are the lo and hi bytes of the start addresses of the pixel lines. What remains is adding the horizontal character position.



hi.jpg (11.48 KiB)

Last edited by fschuhi on Thu Mar 21, 2019 8:48 am, edited 1 time in total.



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Thu Mar 21, 2019 8:30 am

A milestone – well done!



White Flame

Posts: 704

Re: reverse engineering Robotron 2084 for the Apple II

by **White Flame** » Thu Mar 21, 2019 7:48 pm

> **fschuhi wrote:**
>
> I've managed to understand some important subroutines completely, on my own, for the first time

Joined: 24 Jul 2012

> ever
>
> (Don't worry, I won't bother you every time when I understand anything new, it's just a heads-up)

Congratulations! I certainly don't mind you spewing your discoveries on this thread; it's basically your thread, and a nice log for posterity as others might be doing the same thing. I didn't spend that much time in the code, so it's good to see that you're not hitting any barriers in tackling it for real. As a port, hopefully it'll not be super tied into any hardware specifics, though it doesn't look like the Apple II has too many hardware specifics to begin with.

WFDis Interactive 6502 Disassembler
AcheronVM: A Reconfigurable 16-bit Virtual CPU for the 6502 Microprocessor

Post Reply 98 posts 1 ... 3 4 5 6 7

Return to "Nostalgia" Jump to

Forum Home Search Contact

All times are UTC

reverse engineering Robotron 2084 for the Apple II



Chromatix

Posts: 1462
Joined: 21 May 2018

Re: reverse engineering Robotron 2084 for the Apple II

by **Chromatix** » Fri Mar 22, 2019 7:44 am

Interesting to see LDA/PHA/LDA/PHA/RTS used here, at $51C4. A more compact alternative might be:

CODE: SELECT ALL

```
JSR readNextChar   ; just to increment the pointer to
JMP ($0022)   ; indirect through the pointer
```

...which takes 6 bytes instead of 7, not a big difference, and the readNextChar routine isn't very fast. But it does save a bit of explicit stack manipulation.



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Fri Mar 22, 2019 5:10 pm

> **BigEd wrote:**
> A milestone – well done!

Thank you, BigEd, your encouragements are very motivating!



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Fri Mar 22, 2019 5:15 pm

> **Quote:**
>
> As a port, hopefully it'll not be super tied into any hardware specifics, though it doesn't look like the Apple II has too many hardware specifics to begin with. :)

Thanks for the feedback, White Flame. Yes, the Apple II is really a clean platform, perfect for learning 6502. The one thing which I haven't cared about at all yet is the speaker. That's really so basic that it hurts :)



whartung

Posts: 1004
Joined: 13 Dec 2003

Re: reverse engineering Robotron 2084 for the Apple II

by **whartung** » Fri Mar 22, 2019 5:16 pm

> **fschuhi wrote:**
>
> The one thing which I haven't cared about at all yet is the speaker. That's really so basic that it **Hz** :)

Fixed that for ya!

:)



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Fri Mar 22, 2019 5:18 pm

> **Chromatix wrote:**
>
> Interesting to see LDA/PHA/LDA/PHA/RTS used here, at $51C4. A more compact alternative might be:
>
> CODE: SELECT ALL
>
> ```
> JSR readNextChar   ; just to increment the pointer
> JMP ($0022)   ; indirect through the pointer
> ```
>
> ...which takes 6 bytes instead of 7, not a big difference, and the readNextChar routine isn't very fast. But it does save a bit of explicit stack manipulation.

There is not a single indirect jump in the whole Robotron code. I wonder why, seeing how elegant it can be used. Do you use indirect JMP regularly?



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Fri Mar 22, 2019 5:20 pm

> **whartung wrote:**
>
> > **fschuhi wrote:**
> >
> > The one thing which I haven't cared about at all yet is the speaker. That's really so basic that it **Hz** 😀
>
> Fixed that for ya!
> 😀

😁 😁 😁



Chromatix

Posts: 1462
Joined: 21 May 2018

Re: reverse engineering Robotron 2084 for the Apple II

by **Chromatix** » Sat Mar 23, 2019 4:55 am

It is one of the more obscure addressing modes (absolute indirect), as on the NMOS 6502 it's *only* available for JMP. But it's a central part of the Acorn MOS, as each entry point (eg. JSR $FFEE) is often just a JMP ($02xx) through the patch table. It just struck me as the natural way to implement a return from an inline-data subroutine.



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Mon Apr 01, 2019 3:20 pm

Robotron has a great gameplay. It's getting crowded pretty quickly - that's a strength, but it has a steep learning curve. There are distinct types of levels (called "waves" in Robotron), regarding the type of robot enemies you have to battle. The waves get increasingly difficult, fast. In the "Tank Wave" (first occuring as #7) there are Quarks pulsing around and spanning multiple tanks, each shooting bouncing shells https://youtu.be/t7_hvzIaqxk?t=134, that's a good player, still struggling through #7.

Problem: It takes time to get to this level - - no savepoints, no way to train a particular wave. I therefore

inserted a screen to the configuration menu where you can select the wave you want to start with:



The game continues to the controls menu:



The game then starts with the Tank Wave, score is 0:



Robotron cycles through the types while throwing in some additional robot types and generally ramping up the total number of enemies. Here is Tank Wave #27, with two additional Spheriods lurking in the corners:





fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Mon Apr 01, 2019 3:39 pm

I wanted to enter two-digit wave numbers, with backspace, return and escape, handle special cases like "00". It's implemented as a statemachine using an RTS jump table as described in https://wiki.nesdev.com/w/index.php/RTS_Trick. Apart from the special characters, only numbers are allowed. The tutorial http://6502.org/tutorials/compare_instructions.html was very helpful to get the constraint right (I hope).

CODE: SELECT ALL

```
next_char:                      ; main loop
        JSR show_digits
        JSR waitKbdControls

        CMP ch_escape           ; <Escape> breaks
        BEQ break

        CMP ch_return           ; <Return> is a valid
        BEQ .send_event

        CMP ch_left             ; <Left> removes last
        BEQ .send_event

        check_is_number         ; only chars <0> to <
        BCC next_char           ; carry cleared if no

    .send_event:
        STA event               ; dispatch event
        JSR dispatch_event

        LDA state               ; check if we should
        CMP s_exit
```

```
        BEQ exit

        JMP next_char

dispatch_event:
        LDA state               ; state index into RT
        ASL                     ; table has 16bit add
        TAX
        LDA rts_table+1,X       ; push hibyte first
        PHA
        LDA rts_table,X         ; push lobyte second,
        PHA
        LDA event               ; state subroutines g
        RTS

state_hasNone:
        ...
        RTS

state_hasOne:
        ...
        RTS

state_hasTwo:
        ...
        RTS

state_exit:
        ...
        RTS

rts_table:
        .word   state_hasNone-1
        .word   state_hasOne-1
        .word   state_hasTwo-1
```

ATTACHMENTS

hook.asm
(5.37 KiB)



barrym95838

Posts: 2061
Joined: 30 Jun 2013
Location: Sacramento, CA, USA

Re: reverse engineering Robotron 2084 for the Apple II

by **barrym95838** » Tue Apr 02, 2019 3:03 am

I can see the 1 MHz Apple][start to "run out of breath" on some of the busier screens, but it still impresses me that games like this can remain playable and entertaining on hardware with "unassisted" graphics, one-bit sound, '555 A to D, and no interrupts. The "centipede levels" in ***Bandits*** were especially marvelous to me. Benny Ngo was (is?) quite the 6502 virtuoso.

Got a kilobyte lying fallow in your 65xx's memory map? Sprinkle some ***VTL02C*** on it and see how it grows on you!

Mike B. ***(about me)*** ***(learning how to github)***



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Tue Apr 02, 2019 7:02 am

Yes, both Robotron and Bandits have aged very well, IMHO. Apart from the fun gameplay, Bandits is also a formidable piece of software, given what Benny Ngo was able to cram into 6kb.



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

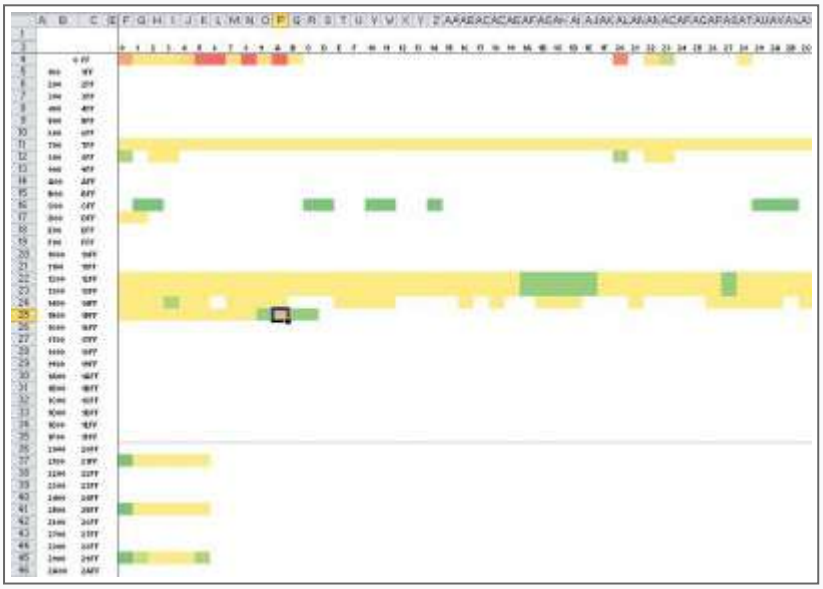
by **fschuhi** » Tue Apr 02, 2019 9:48 pm

I have started to get a closer look at what's happening in terms of reads from and writes to memory. The problem is of course not collecting the info but rather condensing it sensibly. I first tried to group the access in pages, for the different subroutines as suggested by sark02 earlier in this thread. The info is of course helpful but only on a per-subroutine basis. Given the overwhelming number of instructions (and even subroutines) in Robotron, I decided to try to see the bigger picture first, on a per-byte basis.

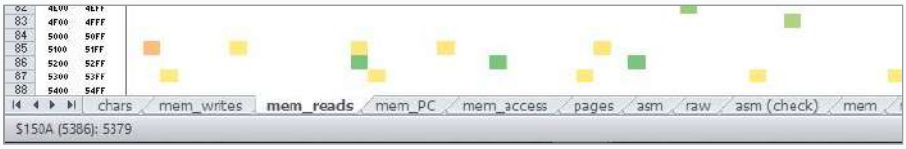
It's work in progress so it's a bit premature to share it here, but maybe you guys have feedback to offer on how to progress from here. That's the reads from a couple of seconds of the intro (up to $CFFF, i.e. including the softswitches but excluding the higher parts of the ROM):



That's really the big picture - - a heatmap with all reads, each line of blocks a page. I can zoom in:



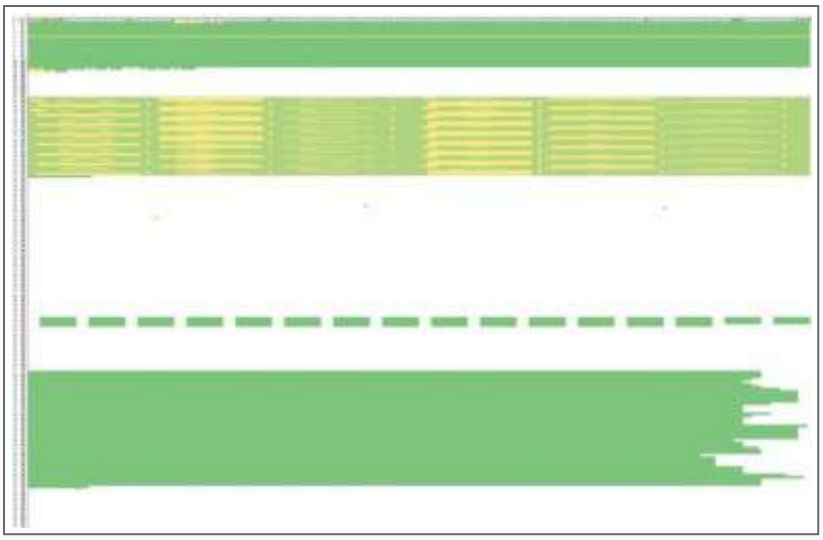
Address and access count of the selected block is shown in the status bar:



reads bottom.jpg (11.33 KiB)

Beginning $2000 is the memory which is mapped to the screen. I had first assumed that I should not see any reads here, only writes, but you can see the effect of the code updating "sprites" in memory. The algorithm obviously needs to know what's in a particular area of the screen to remove and display pixels correctly. Related to displaying graphics are the pages $1200 and $1300. Access to the pages have the same color (i.e. exactly the same counts in both pages for each byte). These are the lobyte/hibyte lookup tables to translate logical pixel line into the where those lines physically reside in memory.

Here the writes which clearly show the hires memory:



There are also some lone hot spots below the hires memory. That's where the code resides, so those bytes are modified by and in the code. The two consecutive ones on the left are part of the printChar subroutine explained in one of my earlier posts.

At last the counts for particular program counters for read and write instructions (hotter colors = instruction was executed more often):



PCs small.jpg (11 KiB)

The workbench is reasonably fast enough to allow "slicing" the total dataset into smaller pictures = containing less cycles. I should thus be able to drill down to seeing a particular sprite drawn on the screen (write) and relate the bitmap to it's origin (read).



Re: reverse engineering Robotron 2084 for the Apple II

by **whartung** » Tue Apr 02, 2019 10:25 pm

The heat map of memory access is a pretty neat feature.

whartung

Posts: 1004
Joined: 13 Dec 2003



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Tue Apr 30, 2019 3:05 pm

After some time away I've been able to spend some more hours on the reverse engineering project.

The Apple II is severly constrained in terms of memory. Consequently, Robotron uses for the "cut scenes" the actual game code to display elements like the player character or enemies. From a reverse engineering point of view this is fortunate, because I can use the uncluttered (and even scripted) movements of elements on the start screens to understand how the game moves "sprites" around the screen. For an example see the movements of the player character on the screen explaining the controls.

I've extended the read/write tracking of the workbench from my last post to project what's happening in memory onto the simulated screen. In the following pic you can see the "echo" of the beginning of the movement of the player character on the controls screen (i.e. what you see in the Youtube video, the character going from left to right first.)



Following the strategy suggested in my last post....

> **Quote:**
>
> The workbench is reasonably fast enough to allow "slicing" the total dataset into smaller pictures = containing less cycles. I should thus be able to drill down to seeing a particular sprite drawn on the screen (write) and relate the bitmap to it's origin (read).

... I can visually do a trial-and-error to constrain the shown range of cycle counts (which I use as global clock) until I only see one particular "frame" of the movement:



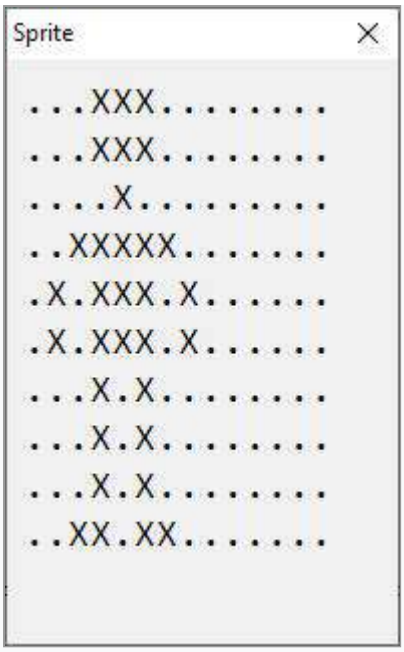
control_single.jpg (3.78 KiB)

The workbench can now show me the corresponding reads from memory:



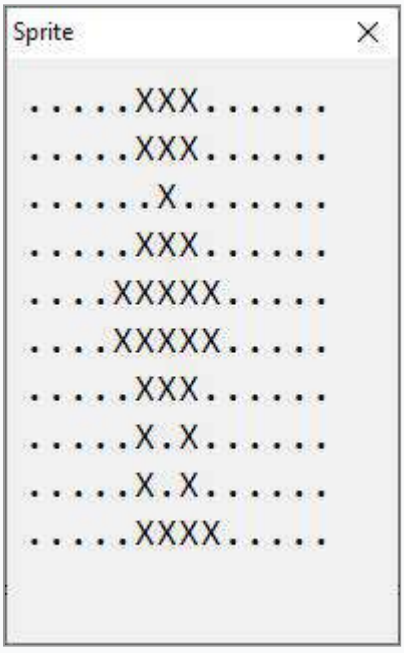
control_reads.jpg (5.17 KiB)

The area between the horizontal lines is the video memory. The colors on the zero page (first line) show that there is some heavy indirect reading involved.
Looking at which code was executed between the shown cycle counts is of course necessary to understand what e.g. the memory related to the two green lines on the lower left contain: It's the actual sprite data, both for removing the first shown player character ("facing the camera")...



man1.jpg (9.57 KiB)

... as well as one frame of the player character in movement to the right:



man2.jpg (9.39 KiB)

From inspecting the code and what's in memory I've gleaned some first information about how the sprites are stored in memory (resulting in the crude X. displays above), in particular how multiple sprites are stored in order to make it easy to horizontally move the sprite pixel-wise through the bits in screen memory.

This part of Robotron is quite central to the overall gameplay. The code takes generalised sprites (i.e. different heights, widths and "facings") and projects them onto the screen, inside bytes and scattered across the video memory (due to Apple II ideosyncrasies). In addition, the code needs to track the state of the sprite, to cycle through e.g. the left+right movements of the legs (or

pulsing shapes etc.), which also needs to take into account the direction of movement. The algorithm is much more involved than I had originally thought it would be. A nice challenge! 😀



fschuhi

Posts: 38
Joined: 23 Feb 2019

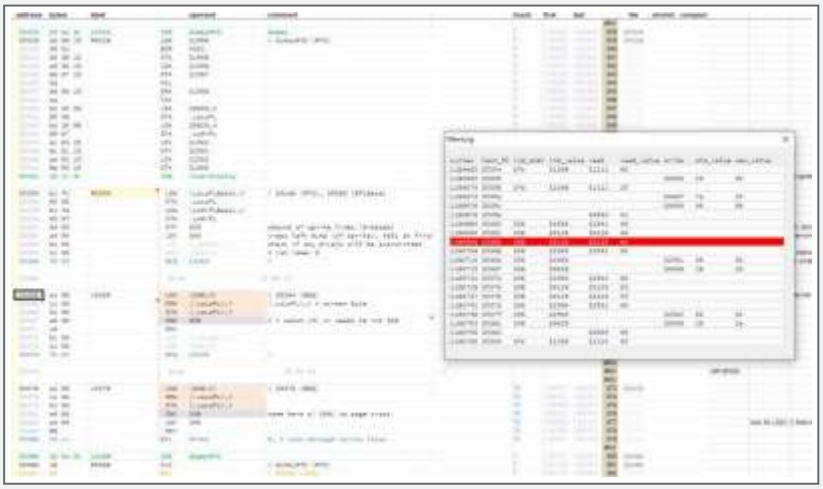
Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Tue Apr 30, 2019 3:50 pm

Having a *good idea* about how something works is still far from really *knowing* the details of how this is accomplished (oh yeah, Captain Obvious, tell us more 😉). The "sprite projector" hinted at in the post above can serve as a good training ground to learn to tear into asm code.

Sadly (or interestingly, actually), my usual debugging skills can not so skillfully employed in the 6502 / Apple II arena. For the Apple II there are of course debuggers with single-stepping and breakpoints, but I think at this point of the learning curve I need more of an "explorer" than a "debugger".

My workbench has the ability to track all memory reads and writes, both absolute and indirect. That's e.g. what's behind the heat maps and the "screen echos" shown in my last two posts. For each read/write I track the cycle count of that operation, together with its PC. For the writes I store both the old value before the operation and the new value stored by the write. Although I do not yet log AXY (and SP only indirectly via the activity on page $01), some good understanding can be derived from relating changes in memory with the flow of instructions.



The pic above shows a part of the "sprite projector" and the memory log, positioned at a particular cycle and PC. I can move upwards and downwards in the MemLog window. This automatically synchronises with the asm

code in the main window. I can also search forward and backward to the same PC, to compare changes in memory between different runs through the same stretches of code. I can also go in the main asm window to any PC and synchronise the MemLog to the first (or nth, forward and backward) occurance.

It's a bit like time travel debugging. For fully fledged time travel I would also have to store all changes to the registers, but that hasn't been necessary so far. Given the importance of the registers for, well, everything, that might come as a surprise. But because registers have to be loaded from somewhere and because this somewhere is usually not far from the point of interest, it's actually quite easy to trace the flow of data both through the memory and also through the registers. I'll see how far I get with that approach.

Post Reply 98 posts 1 ... 3 4 5 6 7

Return to “Nostalgia” Jump to

Forum Home Search Contact

All times are UTC

6502.org is an ongoing project by Mike Naberezny and contributors. About · News · GitHub

reverse engineering Robotron 2084 for the Apple II



Chromatix

Posts: 1462
Joined: 21 May 2018

Re: reverse engineering Robotron 2084 for the Apple II

by **Chromatix** » Fri Jun 14, 2019 6:07 pm

Are your reverse-engineering tools available anywhere? I might try them out on the ROMs from a classic chess computer.



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Fri Jul 19, 2019 1:15 pm

I'm sorry for having been away from the forum for so long, please excuse the late answer.

> **Chromatix wrote:**
>
> Are your reverse-engineering tools available anywhere? I might try them out on the ROMs from a classic chess computer.

Thanks for asking, no, I've not yet been able to upload anything to GitHub or the like, the workbench and it's various tools is much too disjointed to be of any use to anyone. I'll have to do a refactoring anyways as soon as I come back to the project (hopefully in September), so I might package it a bit better and upload it then.

Re: reverse engineering Robotron 2084 for the Apple II

by **Chromatix** » Fri Jul 19, 2019 3:16 pm

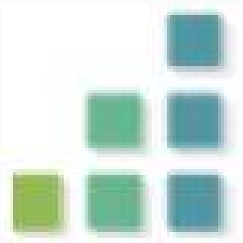


Chromatix

Posts: 1462
Joined: 21 May 2018

Sure, there's no hurry.

I actually found some documentation about the chess engine design used by that particular author, and the small amount of manual analysis I managed to do seems to match certain details of that. It also looks like he's stuck to NMOS coding style, even though the machine uses a CMOS 6502.



cjs

Posts: 759
Joined: 01 Dec 2018
Location: Tokyo, Japan
Contact:

Re: reverse engineering Robotron 2084 for the Apple II

by **cjs** » Wed Oct 30, 2019 10:19 am

I don't know if you're still working on this, many months later, but this thread certainly was an interesting read! I've got a couple of minor comments:

> **fschuhi wrote:**
>
> I've managed to understand some important subroutines completely, on my own, for the first time ever....

CODE: SELECT ALL

```
$51e8  readNextChar  INC $22          ; next char
$51ea                BNE L51ee
$51ec                INC $23          ; stash car

$51ee  L51ee         LDY #$00
$51f0                LDA ($22),Y      ; load char
$51f2                RTS
```

That increment of the high byte is vital, and nothing to do with the length of the stash. If the stash crosses a page boundry, even if it's only two bytes long, the low byte will wrap around to 0 and, if the high byte is not incremented when that happens, you'll end up reading from an address 256 bytes less than the one you should be reading from. (And what you see above is of course the common idiom for 16-bit increments.)

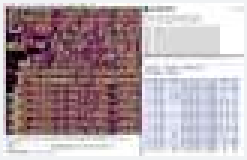
> **Quote:**
>
> I've not yet been able to upload anything to GitHub or the like, the workbench and it's various tools is much too disjointed to be of any use to anyone. I'll have to do a refactoring anyways as soon as I come back to the

> project (hopefully in September), so I might package it a bit better and upload it then.

I wouldn't worry too much about the disjointedness or quality of the code; just get it in the repo, make sure there's a couple of lines of README that say what the repo is and link back to this thread, and worry about cleaning it up later. Having anything at all available for others to see is still much better than having nothing. And of course it serves as a backup as well in case your disk gets trashed or whatever.

Also, what can help with this is always to start out with an empty repo when you start on a project like this and commit at regular checkpoints when you've got something working. This not only lets you easily go back if you mess something up, and review your changes, both of which are useful even if nobody else ever sees the repo, but tends to make one (or me, at least) slightly more organized about what I'm doing. (In particular, this can make a project a *lot* easier to come back to after a couple of weeks or more off.)

Curt J. Sampson - github.com/0cjs



BigEd

Posts: 11585
Joined: 11 Dec 2008
Location: England
Contact: 💬

Re: reverse engineering Robotron 2084 for the Apple II

by **BigEd** » Wed Oct 30, 2019 10:29 am

(Welcome, cjs, good to see your first post!)



fadden

Posts: 2
Joined: 31 Mar 2020

Re: reverse engineering Robotron 2084 for the Apple II

by **fadden** » Tue Mar 31, 2020 4:43 am

There are a number of Apple II game disassemblies collected at https://6502disassembly.com/ (Space Eggs, Stellar 7, Elite, Caverns of Freitag, others). If you're not familiar with common ways to handle the hi-res screen, reading other people's code can be useful.

All of the disassemblies were done with SourceGen (https://6502bench.com/), a fully interactive 6502

disassembler that runs in Windows. I can highly recommend it, both as a heavy user and as the program's author. Being able to decode graphics and embed them in the listings was hugely beneficial when digging through Space Eggs (video: https://youtu.be/lSvEr5nCHbY). SourceGen is free and open-source.



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Mon Jun 08, 2020 1:16 pm

cjs + fadden, thanks for adding posts to this dormant thread!

I intend to come back to the project, not soon but eventually, hopefully in the fall. I have a long reading list and also lined up a number of 6502 tutorials, so I should be able to make a bit progress when coming back to Robotron.



fschuhi

Posts: 38
Joined: 23 Feb 2019

Re: reverse engineering Robotron 2084 for the Apple II

by **fschuhi** » Tue Jun 30, 2020 7:25 pm

I've managed to upload the Python stuff covered in this thread to GitHub:
https://github.com/fschuhi/Robotron_2084

It has been a while (actually more than 1 year) since I've suspended working on the project. It was never discontinued, though, just waited to be rekindled

I did some checks on the Python code today, but I've actually just added it for reference purposes. The main work currently goes into working with C#, namely Sean Fausett's https://github.com/digital-jellyfish/Virtu, more precisely the hex-ray branch of his collaborator Nick Westgate (https://github.com/sicklittlemonkey/Virtu/tree/hex-ray). The hex-ray feature is explained by Nick on https://www.youtube.com/watch?v=Ux4n5Z5E-Lo. This is pretty much what I did on Excel, but in real-time. I'll hope to make use of this feature in my Robotron project.

Nick is a frequent contributor to comp.sys.apple2 and the author of a mod of Robotron 2084 on the Apple (for 2 joysticks). He actually provided me with a partial

disassembly of Robotron which I was able to compare with what I had – – a nice booster for the project!

Even before getting more those additional pointers into the code I had been making much progress using Andy McFadden's SourceGen (see the lasts posts here). With the Python workbench I did only disassembling along the execution path. As some of you pointed out it can be difficult to understand the workings of the code. Working towards a complete disassembly with SourceGen has brought me up to speed again.

Looking back at this thread, I have to say "thank you" once again to all of you folks here. Your comments, code snippets, and views on reengineering and 6502 programming helped to build momentum which was so great that it actually carried the project across a chasm of 2 years. I'm really grateful for your help.

Over the coming weeks (probably well into the fall) I will have to spend time on a day job which consumes much of my bandwith. Robotron will have to be sidelined, but I'll keep working on it from time to time as a nice distraction

Post Reply 98 posts 1 ... 3 4 5 6 7

Return to “Nostalgia” Jump to

Forum Home Search Contact

All times are UTC

6502.org is an ongoing project by Mike Naberezny and contributors. About · News · GitHub